Student of Games: A unified learning algorithm for both perfect and imperfect information games

Martin Schmid[1,2], Matej Moravčík[1,2], Neil Burch[2,3,4], Rudolf Kadlec[1,2], Josh Davidson[2,3], Kevin Waugh[2,3], Nolan Bard[2,3], Finbarr Timbers[2,5], Marc Lanctot[2,6], G. Zacharias Holland[2,3], Elnaz Davoodi[2,6], Alden Christianson[2,7], Michael Bowling[2,4,7*]

[1]EquiLibre Technologies, Prague, Czechia, [2]Google DeepMind,
[3]Sony AI, New York, NY, USA, [4]Amii, Edmonton, Canada,
[5]Midjourney, South San Francisco, CA, USA,
[6]Google DeepMind, Montreal, Canada, [7]University of Alberta, Edmonton, Canada
*To whom correspondence should be addressed; E-mail: mbowling@ualberta.ca

Teaser: Student of Games combines search, learning, and game-theoretic reasoning to play chess, go, poker, and Scotland Yard.

Games have a long history as benchmarks for progress in artificial intelligence. Approaches using search and learning produced strong performance across many perfect information games, and approaches using game-theoretic reasoning and learning demonstrated strong performance for specific imperfect information poker variants. We introduce Student of Games, a general-purpose algorithm that unifies previous approaches, combining guided search, self-play learning, and game-theoretic reasoning. Student of Games achieves strong empirical performance in large perfect and imperfect information games — an important step towards truly general algorithms for arbitrary environments. We prove that Student of Games is sound, converging to perfect play as available computation and approximation capacity increases. Student of Games reaches strong performance in chess and Go, beats the strongest openly available agent in heads-up no-limit Texas hold'em poker, and defeats the state-of-the-art agent in Scotland Yard, an imperfect information game that illustrates the value of guided search, learning, and game-theoretic reasoning.

Introduction

In the 1950s, Arthur L. Samuel developed a checkers-playing program that employed what is now called minimax search (with alpha-beta pruning) and "rote learning" to improve its evaluation function via self-play (*1*). This investigation inspired many others, and ultimately Samuel co-founded the field of artificial intelligence (*2*) and popularized the term "machine learning". A few years ago, the world witnessed a computer program defeat a long-standing professional at the game of Go (*3*). AlphaGo also combined learning and search. Many similar achievements happened in between, such as the race for super-human chess leading to DeepBlue (*4*) and TD-Gammon teaching itself to play master-level performance in Backgammon through self-play (*5*), continuing the tradition of using games as canonical markers of mainstream progress across the field.

Throughout the stream of successes, there is an important common element: the focus on a single game. Indeed, DeepBlue could not play Go, and Samuel's program could not play chess. Likewise, AlphaGo could not play chess; however its successor AlphaZero (*6*) could, and did. AlphaZero demonstrated that a single algorithm could master three different perfect information games — where the game's state is known to all players — using a simplification of AlphaGo's approach, and with minimal human knowledge. Despite this success, AlphaZero could not play poker, and the extension to imperfect information games was unclear.

Meanwhile, approaches taken to achieve super-human poker AI were substantially different. Strong poker play has relied on game-theoretic reasoning to ensure that private information is concealed effectively. Initially, super-human poker agents were based primarily on computing approximate Nash equilibria offline (*7*). Search was then added and proved to be a crucial ingredient to achieve super-human success in no-limit variants (*8–10*). Training for other large games have also been inspired by game-theoretic reasoning and search, such as Hanabi (*11, 12*), The Resistance (*13*), Bridge (*14*), AlphaStar (*15*), and (no-press) Diplomacy (*16–18*). Here again, however, despite remarkable success: each advance was still on a single game, with some clear uses of domain-specific knowledge and structure to reach strong performance.

In this paper, we introduce Student of Games (SoG), an algorithm that generalizes the class of games in which strong performance can be achieved using self-play learning, search, and game-theoretic reasoning. SoG uses growing-tree counterfactual regret minimization (GT-CFR): an anytime local search that builds subgames non-uniformly, expanding the tree toward the most relevant future states while iteratively refining values and policies. In addition, SoG employs sound self-play: a learning procedure that trains value-and-policy networks using both game outcomes and recursive sub-searches applied to situations that arose in previous searches.

Student of Games achieves strong performance in multiple challenge domains with both perfect and imperfect information – an important step towards truly general algorithms that can learn in arbitrary environments. Applications of traditional search suffer well-known problems in imperfect information games (*2, Section 5.6.2*). Evaluation has remained focused on single domains (e.g. poker) despite recent progress toward sound search in imperfect information games (*8, 19, 20*). Student of Games fills this gap, using a single algorithm with minimal

domain-specific knowledge. Its search is sound (*20*) across these fundamentally different game types: it is guaranteed to find an approximate Nash equilibrium by re-solving subgames to remain consistent during online play, and yields low exploitability in practice in small games where exploitability is computable. SoG demonstrates strong performance across four different games: two perfect information (chess and Go) and two imperfect information (poker and Scotland Yard). Finally, unlike poker, Scotland Yard has substantially longer search horizons and game lengths, requiring long-term planning.

Background and Terminology

Student of Games will be presented using the Factored-Observation Stochastic Games (FOSG) formalism. For further details on the formalism, see (*21, 22*).

A game between two players starts in a specific world state w^{init} and proceeds to the successor world states $w \in \mathcal{W}$ as a result of players choosing actions $a \in \mathcal{A}$ until the game is over when a terminal state is reached. A world state can be categorized as a decision node, a terminal node, or a chance node. At a decision node, player $\mathcal{P}(w)$ acts. A terminal node marks the end of a game where no players act. A chance node is a special node representing a stochastic event, such as a die roll, with a fixed distribution. At any world state w, $\mathcal{A}(w) \subseteq \mathcal{A}$ refers to those actions that are available, or legal, in world state w. Sequences of actions taken along the course of the game are called histories and denoted $h \in \mathcal{H}$, with $h' \sqsubseteq h$ denoting a prefix history (subsequence). At terminal histories, $z \subset \mathcal{H}$, each player i receives a utility $u_i(z)$.

An information state is a state with respect to one player's information. Specifically, $s_i \in \mathcal{S}_i$ for player i is a set of histories that are indistinguishable due to missing information. A simple example is a specific decision point in poker where player i does not know the opponent's private cards; the histories in the information state are different only in the chance event outcomes that determine the opponent's private cards, since everything else is public knowledge. A player i plays a policy $\pi_i : \mathcal{S}_i \to \Delta(\mathcal{A})$, where $\Delta(\mathcal{A})$ denotes the set of probability distributions over actions $\mathcal{A}$. The goal of each player is to find a policy that maximizes their own expected utility.

Every time a player takes an action, each player gets a private observation $\mathcal{O}_{\text{priv}(i)}(w, a, w')$ and a public observation $\mathcal{O}_{\text{pub}}(w, a, w')$ as a result of applying action a, changing the game's state from w to w'. In perfect information games, the public observation contains complete information, i.e., $\mathcal{O}_{\text{pub}}(w, a, w') = (w, a, w')$ making any private observations uninformative. Furthermore, the transition function depends only on the active player's action, i.e., $\mathcal{T}(w, a) = \mathcal{T}(w, a_{\mathcal{P}(w)})$. In contrast, imperfect information games have information asymmetry between players and some players will receive informative private observations. A public state $s_{\text{pub}}(h) \in \mathcal{S}_{\text{pub}}$ is the sequence of public observations encountered along the history h. For example, a public state in Texas hold'em poker is represented by initial public information (stack sizes and antes), the betting history, and any publicly revealed board cards. Let $\mathcal{S}_i(s_{\text{pub}})$ be the set of possible information states for player i given s_{pub}: each information state $s_i \in \mathcal{S}_i(s_{\text{pub}})$ is consistent with public observations in s_{pub} but has different sequences of private observations. Supplementary material shows a full example of a FOSG in Figure S1 and an example public

tree in Figure S2.

Imperfect information games introduce additional complexity, as $\mathcal{S}_i(s_{\text{pub}})$ can now contain multiple information states that the player's policy depends on. For example, in poker the information states would contain the private cards of player i. Since past actions can leak otherwise private information, agents must reason about which information states players could be in to act soundly. A public belief state is defined as $\beta = (s_{\text{pub}}, r)$, where the range (or beliefs) $r \in \Delta(\mathcal{S}_1(s_{\text{pub}})) \times \Delta(\mathcal{S}_2(s_{\text{pub}}))$ is a pair of distributions over possible information states for both players representing the beliefs over information states in s_{pub}. A basic depiction of the various components of a public belief state is depicted in Figure 1.

Suppose players use a policy profile $\pi = (\pi_1, \pi_2)$. Denote the expected utility to player to player i as $u_i(\pi_1, \pi_2)$ and $-i$ as the opponent of player i. A best response to a specific opponent policy π_{-i} is any policy π_i^b that achieves maximal utility against π_{-i}: $\pi_i^b \in \{\pi_i \mid u_i(\pi_i, \pi_{-i}) = \max_{\pi_i'} u_i(\pi_i', \pi_{-i})\}$. A policy profile π is a Nash equilibrium if and only if π_1 is a best response to π_2 and π_2 is a best response to π_1. There are also approximate equilibria: π is an ϵ-Nash equilibrium if and only if $u_i(\pi_i^b, \pi_{-i}) - u_i(\pi_i, \pi_{-i}) \leq \epsilon$ for all players i.

In two-player zero-sum games, Nash equilibria are optimal because they maximize worst-case utility guarantees for both players. This worst-case utility is unique and is called the game's value. For such games, a standard metric to represent empirical convergence rate is a strategy's exploitability: how much, on average, a player will lose against a best response relative to a Nash equilibrium. For a given policy profile in a two-player zero-sum game $\pi = (\pi_1, \pi_2)$, $\text{EXPLOITABILITY}(\pi) = (\max_{\pi_1'} u_1(\pi_1', \pi_2) + \max_{\pi_2'} u_2(\pi_1, \pi_2'))/2$. Also, equilibrium strategies in two-player zero-sum games are interchangeable: if π^A and π^B are Nash equilibria, then (π_1^A, π_2^B) and (π_1^B, π_2^A) are also both equilibria. These properties mean that a Nash equilibrium plays perfect defence: it will not lose on expectation against any opponent, even one that is playing a best response to the Nash equilibrium. If the opponent makes mistakes, then the Nash equilibrium policy can win. Thus, it is reasonable for an agent to compute and play a Nash equilibrium, or an approximation to one with low exploitability.

Although the FOSG formalism generalizes beyond two-player zero-sum games, the theoretical guarantee of Nash equilibria outside of this setting is less meaningful and it is unclear how effective they would be (for example, in games with more than two players). In this work, we focus on the two-player zero-sum setting. To put SOG in context, we begin with a high-level description of several techniques that have been dominant in this setting, and then contrast SOG with existing work.

Tree Search and Machine Learning

The first major milestones in the field of game-playing AI were obtained by efficient search techniques inspired by the minimax theorem (*1, 4*). In a two-player zero-sum game with perfect information, the approach uses depth-limited search starting from the current world state w_t, along with a heuristic evaluation function to estimate the value of states beyond the depth limit, $h(w_{t+d})$, and game-theoretic reasoning to back up values (*23*). Researchers developed

notable search enhancements (*24, 25*) that greatly improved performance, leading to IBM's super-human DeepBlue chess program (*4*).

This classical approach was, however, unable to achieve super-human performance in Go, which has substantially larger branching factor and state space complexity than chess. Prompted by the challenge of Go (*26*), researchers proposed Monte Carlo tree search (MCTS) (*27, 28*). Unlike minimax search, MCTS builds trees via simulations, starting with an empty tree rooted by w_t and expanding the tree by adding the first state encountered in simulated trajectories that is not currently in the tree, and finally estimating values from rollouts to the end of the game. MCTS led to substantially stronger play in Go and other games (*29*), attaining 6 dan amateur level in Go. However, heuristics leveraging domain knowledge were still necessary to achieve these milestones.

In AlphaGo (*3*), value functions and policies are incorporated, learned initially from human expert data, and then improved via self-play. A deep network approximates the value function and a prior policy helps guide the selection of actions during the tree search. The approach was the first to achieve super-human level play in Go (*3*). AlphaGo Zero removed the initial training from human data and Go-specific features (*30*). AlphaZero reached state-of-the-art performance in chess and Shogi as well as Go, using minimal domain knowledge (*6*).

Student of Games, like AlphaZero, combines search and learning from self-play, using minimal domain knowledge. Unlike MCTS, which is not sound for imperfect information games, SoG's search algorithm is based on counterfactual regret minimization and is sound for both perfect and imperfect information games.

Game-Theoretic Reasoning and Counterfactual Regret Minimization

In imperfect information games, the choice of strategies that arise from hidden information can be crucial to determining each player's expected rewards. Simply playing too predictably can be problematic: in the classic example game of Rock, Paper, Scissors, the only thing a player does not know is the choice of the opponent's action, however this information fully determines their achievable reward. A player choosing to always play one action (e.g. rock) can be easily beaten by another playing the best response (e.g. paper). The Nash equilibrium plays each action with equal probability, which minimizes the benefit of any particular counter-strategy. Similarly, in poker, knowing the opponent's cards or their strategy could yield substantially higher expected reward, and in Scotland Yard, players have a higher chance of catching the evader if their current location is known. In these examples, players can exploit any knowledge of hidden information to play the counter-strategy resulting in higher reward. Hence, to avoid being exploited, players must act in a way that does not reveal their own private information. We call this general behavior game-theoretic reasoning because it emerges as the result of computing (approximate) minimax-optimal strategies. Game-theoretic reasoning has been paramount to the success of competitive poker AI over the last 20 years.

One algorithm for computing approximate optimal strategies is counterfactual regret minimization (CFR) (*31*). CFR is a self-play algorithm that produces policy iterates $\pi_i^t(s, \cdot) \in \Delta(\mathcal{A})$

for each player i at each of their information states s in a way that minimizes long-term average regret. As a result, the (appropriately weighted) average policy over all T iterations $\bar{\pi}^T$ converges to an ϵ-Nash equilibrium at a rate of $O(1/\sqrt{T})$. At each iteration, t, counterfactual values $v_i^t(s,a)$ are computed for each action $a \in \mathcal{A}(s)$ and immediate regrets for not playing a, $r^t(s,a) = v_i^t(s,a) - \sum_{a \in \mathcal{A}(s)} \pi^t(s,a) v_i^t(s,a)$, are computed and tabulated in a cumulative regret table storing $R^T(s,a) = \sum_{t=1}^{T} r^t(s,a)$. A new policy is computed using regret-matching (*32*): $\pi^{t+1}(s,\cdot) = \frac{[R^t(s,\cdot)]^+}{\sum_a [R^t(s,a)]^+}$, where $[x]^+ = \max(x,0)$, if $\sum_a [R^t(s,a)]^+ > 0$, or the uniform distribution otherwise.

CFR$^+$ (*33*) is a successor of CFR that played a key role in solving the game of heads-up limit hold'em poker, the largest imperfect information game to be solved to date (*34*). A key change in CFR$^+$ is a different policy update mechanism, regret-matching$^+$, which defines cumulative values slightly differently: $Q^t(s,a) = (Q^{t-1}(s,a) + r^t(s,a))^+$, and $\pi^{t+1}(s,a) = Q^t(s,a) / \sum_b Q^t(s,b)$.

A common form of CFR (or CFR$^+$) is one that traverses the public tree of public states, rather than the classical extensive-form game tree of world states (and information states). Quantities required to compute counterfactual values, such as each player's probabilities of reaching each information state under their policy (called their range) are maintained as beliefs. Finally, leaf nodes can be evaluated directly using the ranges, chance probabilities, and utilities (often more efficiently (*35*)).

Imperfect Information Search, Decomposition, and Re-Solving

Solution concepts like Nash equilibria and minimax are defined over policy profiles. A player's policy is fixed during play and solely a function of the information state. Search could instead be described as a process, which might return different action distributions at subsequent visits to the same state. That is, when using search the resulting policy can depend on more than the just the information state, such as time-limits, non-deterministic computation, stochastic events from either the game or within the search, or the outcome of other searches. These factors introduce important subtleties such as solution compatibility across different searches (*20*).

CFR has been traditionally used as a game-solving engine, computing entire policies via self-play. Each iteration traverses the entire game tree or a sampled subtree, recursively computing the counterfactual values for an information state from the values of its successor states. Suppose one wanted a policy for a part of the game up to some depth $d > 0$. If there was an oracle to compute the counterfactual values each player would receive at depth d, then each iteration of CFR could be run to depth d and query the oracle to return the values. As a result, the policies would not be available at depths $d' > d$. Summarizing the policies below depth d by a set of values which can be used to reconstruct policies at depth d and beyond is the basis of decomposition in imperfect information games (*36*). A subgame in an imperfect information game is a game rooted at a public state s_{pub}. In order for a subgame to be a proper game, it is paired with a belief distribution r over initial information states, $s \in \mathcal{S}_i(s_{\text{pub}})$. This is a strict generalization of subgames in perfect information games, where every public state has exactly

one information state (which is, in fact, no longer private as a result) and a singleton belief with probability 1 for both players.

Subgame decomposition has been a crucial component of most recent developments of poker AI that scale to large games such as no-limit Texas hold'em (*8–10, 37*). Subgame decomposition enables local search to refine the policy during play analogously to the classical search algorithms in perfect information games and traditional Bellman-style bootstrapping to learn value functions (*8, 13, 37, 38*). Specifically, a counterfactual value network (CVN) represented by parameters $\boldsymbol{\theta}$ encodes the value function $\mathbf{v}_{\boldsymbol{\theta}}(\beta) = \{v_i(s_i)\}_{s_i \in \mathcal{S}_i(s_{\text{pub}}), i \in \{1,2\}}$, where β includes player's beliefs over information states for the public information at s_{pub}. The function $\mathbf{v}_{\boldsymbol{\theta}}$ can then be used in place of the oracles mentioned above to summarize values of the subtrees below s_{pub}. An example of depth-limited CFR solving using decomposition is shown in Figure 2.

Safe re-solving is a technique that generates subgame policies from only summary information of a previous (approximate) solution: a player's range and their opponent's counterfactual values. This is done by constructing an auxiliary game with specific constraints. The subgame policies in the auxiliary game are generated in a way that preserves the exploitability guarantees of the original solution, so they can replace the original policies in the subgame. Thorough examples of the auxiliary game construction are found in (*36*) and (*19, Section 4.1*).

Continual re-solving is an analogue of classical game search, adapted to imperfect information games, that uses repeated applications of safe re-solving to play an episode of a game (*8*). It starts by solving a depth-limited game tree rooted at the beginning of the game, and search is a re-solving step. As the game progresses, for every subsequent decision at some information state s_i, continual re-solving will refine the current strategy by re-solving at s_i. Like other search methods, it is using additional computation to more thoroughly explore a specific situation encountered by the player.

Related Work

SOG combines many elements that were originally proposed in AlphaZero and its predecessors, as well as DeepStack (*3, 6, 8, 30*). Specifically, SOG uses the combined search and learning using deep neural networks from AlphaGo and DeepStack, along with game-theoretic reasoning and search in imperfect information games from DeepStack. The use of public belief states and decomposition in imperfect information games has been a critical component of success in no-limit Texas Hold'em poker (*8–10, 19, 36, 37, 39*). The main difference from AlphaZero is that the search and self-play training in SOG are also sound for imperfect information games, and evaluation across game types. The main difference from DeepStack is the use of substantially less domain knowledge: the use of self-play (rather than poker-specific heuristics) to generate training data and a single network for all stages of the game. The most closely related algorithm is Recurrent Belief-based Learning (ReBeL) (*37*). Like SOG, ReBeL combines search, learning, and game-theoretic reasoning via self-play. The main difference is that SOG is based on (safe) continual re-solving and sound self-play. To achieve ReBeL's guarantees,

its test-time search must be conducted with the same algorithm as in training, whereas SoG can use any belief-based value-and-policy network of the form described in Counterfactual Value-and-Policy Networks (similarly to e.g. AlphaZero, which trains using 800 simulations but then can use substantially larger simulation limits at test-time, which is needed for strong performance in many perfect information domains). SoG is also validated empirically across different challenge games of different game types, whereas ReBeL is only evaluated on two imperfect information games.

There has been considerable work in search for imperfect information games. One method that has been quite successful in practice is determinization: at decision-time, a set of of candidate world states are sampled, and some form of search is performed (*40, 41*). In fact, the baseline player we use to compare SoG to in Scotland Yard, PimBot (*42, 43*), is based on these methods and achieved state-of-the-art results. However, these methods are not guaranteed to converge to an optimal strategy over time. We demonstrate this lack of convergence in practice over common search algorithms and standard reinforcement learning (RL) benchmarks in supplementary text. In contrast, the search in SoG is based on game-theoretic reasoning. Other algorithms have proposed adding game-theoretic reasoning to search: Smooth UCT (*44*) combines Upper Confidence Bounds applied to Trees (UCT) (*27*) with fictitious play, however its convergence properties are not known. Online Outcome Sampling (*45*) derives an MCTS variant of Monte Carlo CFR (*46*); however, OOS is only guaranteed to approach an approximate equilibrium at a single information state (local consistency) and has not been evaluated in large games. GT-CFR used by SoG makes use of sound search based on decomposition and is globally consistent (*20, 36*).

There have been a number of RL algorithms that have been proposed for two-player zero-sum games: Fictitious Self-Play (*47*), Policy-Space Response Oracles (PSRO) (*48*), Double Neural CFR (*49*), Deep CFR and DREAM (*50, 51*), Regret Policy Gradients (*52*), Exploitability Descent (*53*), Neural Replicator Dynamics (NeuRD) (*54*), Advantage Regret-Matching Actor Critic (*55*), Friction FoReL (*56*), Extensive-form Double Oracle (XDO) (*57*), Neural Auto-curricula (NAC) (*58*), and Regularized Nash Dynamics (R-NaD) (*59*). These methods adapt classical algorithms for computing (approximate) Nash equilibria to the RL setting with sampled experience and general function approximation. As such, they combine game-theoretic reasoning and learning. Several of these methods have shown promise to scale: Pipeline PSRO defeated the best openly available agent in Stratego Barrage; ARMAC showed promising results on large poker games. R-NaD truly demonstrated scale by obtaining human-level performance in the very large game of Stratego (*59*). In Starcraft, AlphaStar was able to use human data and game-theoretic reasoning to create a master-level real-time strategy policy (*15*). However, none of them can use search at test-time to refine their policy; this shifts a learning and function approximation burden onto training, typically making these methods more computationally demanding in both training time and model capacity to encode a policy or value function.

Lastly, there have been works that use some combination of search, learning, and/or game-theoretic reasoning applied to specific domains. Neural networks have been trained via Q-learning to learn to play Scotland Yard (*60*); however, the overall play strength of the resulting

policy was not directly compared to any other known Scotland Yard agent. In poker, Supremus proposed a number of improvements to DeepStack and demonstrated that they make a big difference when playing human experts (*38*). Another work used a method inspired by DeepStack applied to The Resistance (*13*). In the cooperative setting, several works have made use of belief-based learning (and search) using public subgame decomposition (*12, 61, 62*), applied to Hanabi (*11*). Learning and game-theoretic reasoning were also recently combined to produce agents that play well with humans without human data on the collaborative game Overcooked (*63*). Search and reinforcement learning were combined to produce a bridge bidding player that cooperated with a state-of-the-art bot (WBridge5) and with humans (*14*). Of considerable note is the game of (no-press) Diplomacy. In that game, game-theoretic reasoning was combined with learning in Best Response Policy Iteration (*16*), and game-theoretic search and supervised learning were combined in (*17*) reaching human-level performance on the two-player variant. Recently, all three were combined in DORA (*18*), which learned to play Diplomacy without human data reaching human-level performance on the two-player variant, and subsequently Cicero (*64*) reached human-level on the full game including communication with humans via language models. The main difference between SOG and these works is that they focus on specific games and exploit domain-specific knowledge to attain strong performance.

Descriptions of Challenge Domains

Chess and Go are well-known classic games, both seen as grand challenges of AI (*4, 26*) that have driven progress in artificial intelligence since its inception. The achievement of Deep-Blue beating Kasparov in 1997 is widely regarded to be the first big milestone of AI. Today, chess playing computer programs remain consistently super-human, and one of the strongest and most widely-used programs is Stockfish (*65*). Go emerged as the favorite new challenge domain, which was particularly difficult for classical search techniques (*26*). Monte Carlo tree search (*27–29*) emerged as the dominant search technique in Go. The best of these programs, Crazy Stone and Zen, were able to reach the level of 6 dan amateur (*3*). It was not until 2016 that AlphaGo defeated the first human professional Lee Sedol in the historical 2016 match, and also defeated the top human Ke Jie in 2017.

Heads-up no-limit Texas hold'em is the most common two-player version of poker played by humans, which is also played by DeepStack and Libratus (*8, 9*). Human expert-level poker has been the standard challenge domain among imperfect information games, inspiring the field of game theory itself. No-limit Texas hold'em presents the complexity of stochastic events (card draws), imperfect information (private cards), and a very large state space (*66*). In this paper, we use blinds of 100 and 50 chips, and stack sizes of 200 big blinds (20,000 chips).

Scotland Yard is a compelling board game of imperfect information, receiving a Spiel des Jahres award in 1983 as well as being named the "The most popular game '83" by Spiel-Box (*67*). The game is played on a map of London, where locations are connected by edges representing different modes of transportation. One player plays as "Mr. X" (the evader) and

others control detectives (pursuers). Mr. X is only visible on specific rounds, but detectives get to see the mode of transportation Mr. X uses every round (e.g. taxi, bus, subway). In order to win, detectives need to catch Mr. X within 24 rounds. Scotland Yard is a perfect example of an imperfect information game that requires search for strong play—the detectives have to plan multiple moves into the future while reasoning about possible locations that Mr. X may be. Similarly, though Mr. X has perfect information, he must also reason about where he could be to, for example, avoid revealing his location. Unlike poker, Scotland Yard has partially-observable actions, so private information is effected by the agents' choices in addition to chance.

This suite of games covers the classic challenge domains across game types (perfect information and imperfect information, some with stochastic elements and others not), as well as an additional challenging imperfect information game with substantially longer sequences of actions and a fundamentally different type of uncertainty over hidden actions.

Results

In order to understand the results, we give a brief high-level overview of our main algorithm, Student of Games, which we present formally in the Materials and Methods section below.

Student of Games: Algorithm Summary

The SoG algorithm trains the agent via sound self-play: each player, when faced with a decision to make, employs a sound growing-tree CFR (GT-CFR) search equipped with a counterfactual value-and-policy network (CVPN) to generate a policy for the current state, which is then used to sample an action to take.

GT-CFR grows a tree, starting with the current public state, and consists of two alternating phases: the regret update phase runs public tree CFR updates on the current tree; the expansion phase expands the tree by adding new public states via simulation-based expansion trajectories. One iteration of GT-CFR consists of one run of the regret update phase followed by one run of the expansion phase.

The self-play process generates two types of training data for updating the value and policy networks: search queries, which are public belief states that were queried by the CVPN during the GT-CFR regret update phase, and full-game trajectories from the self-play games. The search queries must be solved to compute counterfactual value targets for updating the value network. The full-game trajectories provide targets for updating the policy network. In practice, the self-play data generation and training happen in parallel: actors generate the self-play data (and solve queries) while trainers learn new networks and periodically update the actors.

Theoretical Results

We have two main theoretical results, which we describe here only informally. They are formally treated in Materials and Methods. Theorem 1 ensures that the exploitability of the final

GT-CFR policy is at most $O(1/\sqrt{T})$, where T is number of GT-CFR iterations, under some conditions on how the search tree is expanded and so long as the value function is reasonably accurate. SoG invokes GT-CFR to re-solve a subtree every time it must act. Thereom 2 bounds the exploitability of the entire SoG policy proving that it is sound to employ GT-CFR recursively. Both theorems together ensure that the search is sound up to some acceptable error in the value function. If there is no error in the value function, and the values are the game-theoretic optimal values, then GT-CFR will provably converge to a Nash equilibrium strategy if run under the conditions stated in the theorems.

Experimental Results

We evaluate SoG on four games: chess, Go, heads-up no-limit Texas hold'em poker, and Scotland Yard. We also evaluate SoG on the commonly-used small benchmark poker game Leduc hold'em, and a custom-made small Scotland Yard map, where the approximation quality compared to the optimal policy can be computed exactly.

When reporting the results we use the notation $\text{SoG}(s, c)$ for SoG running GT-CFR with s total expansion simulations, and c expansion simulations per regret update phase, so the total number of GT-CFR iterations is then $\frac{s}{|c|}$. For example, $\text{SoG}(8000, 10)$ refers to 8000 expansion simulations at 10 expansions per regret update (800 GT-CFR iterations). We choose this notation style to be easily comparable to number of simulations in AlphaZero.

Exploitability in Leduc Poker and Small Scotland Yard Map

Supporting our theoretical results, we empirically evaluate the exploitability of SoG in Leduc poker (*68*) and in Scotland Yard on a small map named "glasses". The full description of Leduc poker is presented in supplementary text and the map is illustrated in Figure S3.

Exploitability is a function of a specific (fixed) policy profile. However, for a search algorithm like SoG, previous searches may affect policies computed at later points within the same game, as explained in Imperfect Information Search, Decomposition, and Re-Solving. Hence, we construct multiple samples of the SoG policy by choosing a random seed, running the search algorithm at every public state in a breadth-first manner such that every search is conditioned on previous searches at predecessor states, and composing together the policies obtained from each search. We then show the minimum, average, and maximum exploitabilities over policies constructed in this way from 50 different choices of seeds. If the minimum and maximum exploitability values are tight, then they represent an accurate estimate of true exploitability.

Figure 3 shows the exploitability of SoG in Leduc poker and the glasses map of Scotland Yard, as a function of the number of CVPN training steps. For these graphs, we evaluate multiple networks (each trained for a different number of steps) generated by a single training run of $\text{SoG}(100, 1)$. Each data point corresponds to a specific network (determined by number of steps trained) being evaluated under different settings during play. For each specific x-value, a single network was used to obtain each exploitability value of SoG using the network under

different evaluation conditions.

We observe that exploitability drops fairly quickly as the training steps increase. Also, even using only 1 CFR update per simulation, there is significant difference in exploitability when more simulations are used. As Theorem 1 suggests, more training (by reducing ϵ) and more search (by increasing T) reduces the exploitability of SoG. Standard RL algorithms in self-play are not guaranteed to reduce exploitability with continued training in this setting. We show this lack of convergence in practice in supplementary text.

Results in Challenge Domains

Our main results compare the performance of SoG to other agents in our challenge domains. We trained a version of AlphaZero using its original settings in chess and Go, e.g. , using 800 MCTS simulations during training, with 3500 concurrent actors each on a single TPUv4, for a total of 800k training steps. SoG was trained using a similar amount of TPU resources.

In chess, we evaluated SoG against Stockfish 8 level 20 (*65*) and AlphaZero. SoG$(400, 1)$ was run in training for 3M training steps. During evaluation, Stockfish uses various search controls: number of threads, and time per search. We evaluate AlphaZero and SoG up to 60,000 simulations. A tournament between all of the agents was played at 200 games per pair of agents (100 games as white, 100 games as black). From this tournament, we rank players according to their Elo ratings. Elo is a classic system for rating chess players originally designed by Arpad Elo in 1967 and still widely-used today (*69*) in many games. A rating, r_i, is assigned to each player i such that a logistic model predicts the probability of player i beating player j as $1/(1 + 10^{(r_j - r_i)/400})$. Table 1 shows the relative Elo comparison obtained by this tournament, where a baseline of 0 is chosen for Stockfish(threads=1, time=0.1s).

In Go, we evaluate SoG$(60000, 10)$ using a similar tournament as in chess, against two previous Go programs: GnuGo (at its highest level, 10) (*70*) and Pachi v7.0.0 (*71*) with 10k and 100k simulations, as well as AlphaZero (*6*) with a range of search simulations at different points in training. SoG$(400, 1)$ was used in training for 1M training steps. Table 1 shows the relative Elo comparison for a subset of the agents that played in this tournament, where a baseline of 0 is chosen for GnuGo. The full results are presented in Tables S3 and S4.

Notice in both chess and Go that SoG reaches strong performance. In chess, SoG$(60000, 10)$ is stronger than Stockfish using 4 threads and one second of search time. In Go, SoG$(16000, 10)$ is more than 1100 Elo stronger than Pachi with 100,000 simulations. Also, SoG$(16000, 10)$ wins 0.5% (2/400) of its games against AlphaZero(s=8000,t=800k). As a result, SoG appears to be performing at the level of top human amateur, possibly even professional level. In both cases, SoG is weaker than AlphaZero, with the gap being smaller in chess. We hypothesize that this difference is the result of MCTS being more efficient than CFR on perfect information games, as the price of SoG's generality.

For chess and Go, we also present direct Elo comparisons from a tournament between AlphaZero (trained for 800k steps) and SoG agents when increasing the number of neural network evaluations in Figure 4. These results demonstrate that SoG is able to scale, improving perfor-

mance with available computation. Note that while the neural networks evaluations account for the majority of the run time, the complexity of the regret update phase is linear in the size of the tree. The run time is thus quadratic in the number of GT-CFR iterations. The absolute time cost could be reduced by an implementation that runs the regret update and expansion phase in parallel. For a more detailed analysis of SoG's complexity, see Performance Guarantees for Continual Re-solving. Intuitively, we would expect $c = 1$ (corresponding to one regret update per expansion simulation) to be best choice. Due to these computational constraints, we chose by hand a small number of values for $c > 1$. Interestingly, we did notice that $c = 1$ is not always the best choice in practice and hope to explore this more thoroughly in the future.

In heads-up no-limit Texas hold'em, we evaluate SoG against Slumbot2019 (*72, 73*), the best open-source heads-up no-limit computer poker player. When training poker, SoG uses randomized betting abstractions described in supplementary text to reduce the number of actions from 20,000 to 4 or 5. SoG$(10, 0.01)$ is trained for up to 1.1M training steps and then evaluated. Since poker has particularly high variance, we use the Action-Informed Value Assessment Tool (AIVAT) (*74*) to compute a more accurate estimate of performance. We also evaluate SoG against a local best-response (LBR) player that can use only fold and call actions with a poker-specific heuristic, which has shown to find exploits in previous poker agents (*75*). Table 2 summarizes the results of SoG along with other recent poker agents. SoG$(10, 0.01)$ wins on average 7 ± 3 milli big blinds (0.7 chips) per hand, with 95% confidence intervals (3.1M matches). LBR fails to find an exploit of SoG's strategy, and SoG wins on average by 434 ± 9 milli big blinds per hand.

In Scotland Yard, the current state-of-the-art agent in this game is based on MCTS with game-specific heuristic enhancements (*42*). We call this agent "PimBot" based on its main author, Joseph Antonius Maria ("Pim") Nijssen. PimBot implements a variant of MCTS that uses determinization, heuristic evaluations and playout policies (*42, 43*). PimBot won 34 out of 50 manually played games against the Nintendo DS Scotland Yard AI.

In our experiment SoG is trained up to 17M steps. In evaluation we play a head-to-head match with SoG$(400, 1)$ against PimBot at different number of simulations per search. The results are shown in Figure 5. These results show that SoG is winning significantly even against PimBot with 10M search simulations (55% win rate), compared to SoG searching a tiny fraction of the game. Interestingly PimBot does not seem to play stronger with more search at this point, as both the 1M and 10M iteration versions have the same performance against SoG.

As in chess and Go, SoG also demonstrates strong performance in these complex imperfect information games. In the case of poker, in addition to beating Slumbot it also beats the local best-response agent which was not possible for some previous agents (including Slumbot). Finally, SoG significantly beats the state-of-the-art agent in Scotland Yard, an imperfect information game with longer episodes and fundamentally different kind of imperfect information than in poker. Together, these results indicate that SoG is capable of strong performance across four games, two fundamentally different game types, and can act as a truly unified algorithm combining search, learning, and game-theoretic reasoning for competitive games.

Discussion

Student of Games (SoG) is a unified algorithm that combines search, learning, and game-theoretic reasoning. SoG is comprised of two main components: a growing-tree counterfactual regret minimization (GT-CFR) technique, and sound self-play which learns counterfactual value-and-policy networks via self-play. Most notably, SoG is a sound algorithm for both perfect and imperfect information games: as computational resources increase, SoG is guaranteed to produce better approximation of minimax-optimal strategies. This finding is also verified empirically in Leduc poker, where additional search leads to test-time approximation refinement, unlike any pure reinforcement learning algorithms that do not use search.

In addition to being sound, SoG also demonstrates strong performance on challenge domains, using minimal domain knowledge. In the perfect information games of chess and Go, SoG performs at the level of human experts or professionals, but can be substantially weaker in head-to-head play than specialized algorithms for this class of games, like AlphaZero, when given the same resources. In the imperfect information game no-limit Texas hold'em poker, SoG beats Slumbot, the best openly available poker agent, and is shown not to be exploited by a local best-response agent using poker-specific heuristics. In Scotland Yard, SoG defeats the state-of-the-art agent.

There are some limitations of SoG that are worth investigating in future work. First, the use of betting abstractions in poker could be removed in favor of a general action-reduction policy for large action spaces. Second, SoG currently requires enumerating the information states per public state, which can be prohibitively expensive in some games; this might be approximated by a generative model that samples world states and operates on the sampled subset. Finally, substantial computational resources are used to attain strong play in challenge domains; an interesting question is whether this level of play is achievable with less computational resources.

Materials and Methods

We now give a detailed description of the Student of Games algorithm. As SoG has several components, we describe them each individually first, and then describe how they are all combined toward the end of the section. For clarity, many of the details (including full pseudocode) are presented in supplementary text.

Counterfactual Value-and-Policy Networks

The first major component of SoG is a counterfactual value-and-policy network (CVPN) with parameters $\boldsymbol{\theta}$, depicted in Figure 6. These parameters represent a function $f_{\boldsymbol{\theta}}(\beta) = (\mathbf{v}, \mathbf{p})$, where outputs $\mathbf{v}$ are counterfactual values (one per information state per player), and prior policies $\mathbf{p}$, one per information state for the acting player, in the public state $s_{\text{pub}}(h)$ at some history of play h.

In our experiments, we use standard feed-forward networks and residual networks. The details of the architecture are described in supplementary text.

Search via Growing-Tree CFR

Growing-tree CFR (GT-CFR) is an algorithm that runs a CFR variant on a public game tree that is incrementally grown over time. GT-CFR starts with an initial tree, $\mathcal{L}^0$, containing β and all of its child public states. Then each iteration, t, of GT-CFR consists of two phases:

1. The regret update phase runs several public tree CFR updates on the current tree $\mathcal{L}^t$.
2. The expansion phase expands $\mathcal{L}^t$ by adding new public states via simulation-based expansion trajectories, producing a new larger tree $\mathcal{L}^{t+1}$.

When reporting the results we use the notation $\text{SoG}(s, c)$ for SoG running GT-CFR with s total expansion simulations, and c expansion simulations per regret update phase, so the total number of GT-CFR iterations is then $\frac{s}{\lceil c \rceil}$. The c can be fractional, so e.g. 0.1 indicates a new node every 10 regret update phases. Figure 7 depicts the whole GT-CFR cycle. We chose this specific notation to directly compare total expansion simulations, s, to AlphaZero.

The regret update phase runs $\left\lceil \frac{1}{c} \right\rceil$ updates (iterations) of public tree CFR on $\mathcal{L}^t$ using simultaneous updates, regret-matching$^+$, and linearly-weighted policy averaging (*33*). At public tree leaf nodes, a query is made to the CVPN at belief state β', whose values $f_{\boldsymbol{\theta}}(\beta') = (\mathbf{v}, \mathbf{p})$ are used as estimates of counterfactual values for the public subgame rooted at β'.

In the expansion phase, new public tree nodes are added to $\mathcal{L}$. Search statistics, initially empty, are maintained over information states s_i, accumulated over all expansion phases within the same search. At the start of each simulation, an information state s_i is sampled from the beliefs in β_{root}. Then, a world state w_{root} is sampled from s_i, with associated history h_{root}. Actions are selected according to a mixed policy that takes into account learned values (via $\pi_{\text{PUCT}}(s_i(h))$) as well as the currently active policy ($\pi_{\text{CFR}}(s_i(h))$) from search: $\pi_{\text{select}}(s_i(h)) = \frac{1}{2}\pi_{\text{PUCT}}(s_i(h)) + \frac{1}{2}\pi_{\text{CFR}}(s_i(h))$. The first policy is determined by PUCT (*3*) using counterfactual values $v_i(s_i, a)$ normalized by the sum of the opponent's reach probability at s_i to resemble state-conditional action values, and the prior policy $\mathbf{p}$ obtained from the queries. The second is simply CFR's average policy at $s_i(h)$. As soon as the simulation encounters an information state $s_i \in s_{\text{pub}}$ such that $s_{\text{pub}} \notin \mathcal{L}$, the simulation ends, s_{pub} is added to $\mathcal{L}$, and visit counts are updated along nodes visited during the trajectory. Similarly to AlphaZero (*6*), virtual losses (*76*) are added to the PUCT statistics when doing $\lceil c \rceil$ simulations inside one GT-CFR iteration.

AlphaZero always expands a single action/node at the end of the iteration (the action with the highest UCB score). Optimal policies in perfect information games can be deterministic, and expanding a single action/node is a good way to avoid unneeded computation after unpromising actions. MCTS methods are sound as long as the best action has been added, which is always true in the limit as the tree is completely filled out. In imperfect information games, optimal policies might be stochastic, having non-zero probability over multiple actions. Rather than

expanding a single action, SoG thus expands the top k actions as ranked by the prior. We use $k = 1$ for perfect information games, where computation cost is very important and we only need to find a single good action, and $k = \infty$ to add all children for imperfect information games where it is important to mix over multiple actions. In addition to being sound in the limit, SoG also has a finite-time guarantee on policy quality when $k = \infty$.

Modified Continual Re-solving

The continual re-solving method used by DeepStack (*8*) takes advantage of a few poker properties, which are not found in other games like Scotland Yard, so we use a more general re-solving method that can be applied to a broader class of games. Recall that the re-solving step and the corresponding auxiliary game requires i) the current player's range ii) the opponent's counterfactual values. This provides a succinct and sufficient representation to safely re-solve the subgame rooted in a public state s_{pub}. In hold'em poker, players generally take turns making actions, and a depth-limited search tree for a re-solving auxiliary game can always be deep enough to contain a state for the opponent's action. All player actions are fully visible to both players, so the opponent's maximum counterfactual value in the previous search tree can be used for the next re-solving auxiliary game, no matter what opponent action we are responding to. By working within the single, fixed domain of poker, these properties let DeepStack and Libratus simply retrieve the re-solving summary information from its previous search.

As SoG is a general algorithm, it can no longer leverage this special case. The current public state s_{pub} might not have been included in the previous search tree, so the prior computation might not directly provide us with the required summary information for re-solving the subgame rooted in s_{pub}. SoG thus starts its re-solving process in the state closest to the current state that is included in the previous search tree: s_{pub}^{prev}. We initialize the search tree with a single branch leading from $s_{\text{pub}}^{\text{prev}}$ to s_{pub}, with all off-branch actions being leaves. The search tree is then expanded forward from s_{pub}, as in DeepStack or Libratus. This re-solving auxiliary game uses summary information for $s_{\text{pub}}^{\text{prev}}$ instead of s_{pub}, and by construction these values and probabilities are available in the previous search tree.

When generating a policy for this next re-solving auxiliary game with GT-CFR, we constrain the expansion phase of GT-CFR to only grow the tree under s_{pub}, to focus the computation on the states relevant for the current decision. After re-solving, our action probabilities for our current information state will still come from the new re-solved policy at s_{pub}, which might not be at the root of the search tree.

Finally, like DeepStack, the gadget for the re-solving auxiliary game is modified by mixing in the opponent's range from the previous search. As introduced in (*36*), the gadget used to transform a subgame into a re-solving auxiliary game is a binary opponent decision for each opponent information state before the subgame. At each information state, the opponent can either terminate (T) and receive the opponent counterfactual values in the re-solving summary, or follow (F) this line of play into the corresponding subgame. The effect of the gadget is to generate an opponent range r for the subgame. Given an opponent range r^{prev} from the

previous search, (*8*) modified the opponent range to be $\alpha r + (1-\alpha) r^{\text{prev}}$. As with DeepStack, this regularization towards the previous opponent policy empirically improves the performance, and we used $\alpha = 0.5$.

Performance Guarantees for Continual Re-solving

Growing the tree in GT-CFR allows the search to selectively focus on parts of the space that are important for local decisions. Starting with a small tree and adding nodes over time does not have an additional cost in terms of convergence:

Theorem 1. *Let $\mathcal{L}^t$ be the public tree at time t. Assume public states are never removed from the search tree, so $\mathcal{L}^t \subseteq \mathcal{L}^{t+1}$. For any given tree $\mathcal{L}$, let $\mathcal{N}(\mathcal{L})$ be the interior of the tree: all non-leaf, non-terminal public states where GT-CFR generates a policy. Let $\mathcal{F}(\mathcal{L})$ be the frontier of $\mathcal{L}$, containing the non-terminal leaves where GT-CFR uses ϵ-noisy estimates of counterfactual values. Let U be the maximum difference in counterfactual value between any two strategies, at any information state, and A be the maximum number of actions at any information state. Then, the regret at iteration T for player i is bounded:*

$$R_i^{T,\textit{full}} \leq \sum_{t=1}^{T} |\mathcal{F}(\mathcal{L}^t)|\epsilon + \sum_{s_{\text{pub}} \in \mathcal{N}(\mathcal{L}^T)} |\mathcal{S}_i(s_{\text{pub}})| U \sqrt{AT}$$

The regret $R^{T,\text{full}}$ in Theorem 1 is the gap in performance between GT-CFR iterations and the highest-value strategy. Theorem 1 shows that the average policy returned by GT-CFR converges towards a Nash equilibrium at a rate of $1/\sqrt{T}$, but with some minimum exploitability due to ϵ-error in the value function. There is also no additional cost when using GT-CFR as the game-solving algorithm for each re-solving search step in continual re-solving:

Theorem 2. *Assume we have played a game using continual re-solving, with one initial solve and D re-solving steps. Each solving or re-solving step finds an approximate Nash equilibrium through T iterations of GT-CFR using an ϵ-noisy value function, public states are never removed from the search tree, the maximum interior size $\sum_{s_{\text{pub}} \in \mathcal{N}(\mathcal{L}^T)} |\mathcal{S}_i(s_{\text{pub}})|$ of the tree is always bounded by N, the frontier size of the tree is always bounded by F, the maximum number of actions at any information states is A, and the maximum difference in values between any two strategies is U. The exploitability of the final strategy is then bounded by $(5D+2)\left(F\epsilon + NU\sqrt{\frac{A}{T}}\right)$.*

Theorem 2 is similar to Theorem 1 of (*8*), adapted to GT-CFR and using a more detailed error model which can more accurately describe value functions trained on approximate equilibrium strategies. It shows that continual re-solving with GT-CFR has the general properties we might desire: exploitability decreases with more computation time and decreasing value function error, and only increases linearly with game length. Proofs of these theorems are presented as supplementary text.

The computational complexity of a GT-CFR re-solving step with T iterations expanding k children is $\mathcal{O}(kT^2)$ public states visited and CVPN network calls. In the special case of perfect information games, the number of network calls can be reduced to $\mathcal{O}(T)$. At every iteration t, in the expansion phase GT-CFR will traverse a single trajectory through the tree to expand a leaf, and use the CVPN to evaluate the newly expanded children. This requires k network calls, and a worst case of $|\mathcal{L}^t|$ states visited or $\lceil \log_b |\mathcal{L}^t| \rceil$ states visited in a balanced b-ary tree. In the regret update phase, GT-CFR visits every state in $\mathcal{L}^t$ and uses the CVPN to evaluate every leaf of the tree. Because k child states are added to the tree at each iteration, $|\mathcal{L}^t| \leq \mathcal{O}(kt)$, giving the stated bounds.

In perfect information games, $k = 1$, each player range is a single number, and we only need to evaluate a state once because the optimal policy does not depend on the player ranges. If a state is evaluated once with a range of 1 for both players and then stored, any other belief state can by evaluated by scaling the stored result by the opponent's ranges.

Data Generation via Sound Self-play

Student of Games generates episodes of data in self-play by running searches at each decision point. Each episode starts at the initial history h_0 corresponding to the start of the game, and produces a sequence of histories $(h_0, h_1, \cdots)$. At time t, the agent runs a local search and then selects an action a_t, and the next history h_{t+1} is obtained from the environment by taking action a_t at h_t. Data for training the CVPN is collected via resulting trajectories and the individual searches.

When generating data for training the CVPN, it is important that searches performed at different public states be consistent with both the CVPN represented by $\boldsymbol{\theta}$ and with searches made at previous public states along the same trajectory (e.g. two searches should not be computing parts of two different optimal policies). This is a critical requirement for sound search (*8,20,36*), and we refer to the process of a sound search algorithm generating data in self-play as sound self-play. To achieve sound self-play, searches performed during data generation run GT-CFR on the modified safe re-solving auxiliary game (as described in Modified Continual Re-solving).

Training Process

The quality of the policies produced by GT-CFR and data generated by sound self-play depends critically on the values returned by the CVPN. Hence, it is important for the estimates to be accurate in order to produce high-performance searches and generate high-quality data. In this subsection, we describe the procedure we use to train the CVPN. The process is summarized in Figure 8.

Query Collection

As described in Search via Growing-Tree CFR and Data Generation via Sound Self-play, episodes are generated by each player running searches of GT-CFR from the current public

state. Each search produces a number of network queries from public tree leaf nodes β (depicted as pink nodes in Figure 8).

The training process improves the CVPN via supervised learning. Values are trained using Huber loss (*77*) based on value targets and the policy loss is cross entropy with respect to a target policy. Value and policy targets are added to a sliding window data set of training data that is used to train the CVPN concurrently. The CVPN is updated asynchronously on the actors during training.

Computing Training Targets

Policy targets are assembled from the searches started at public states along the main line of episodes (the histories reached in self-play) generated by sound self-play described in Data Generation via Sound Self-play. Specifically, they are the output policies for all information states within the root public state, computed in the regret update phase of GT-CFR.

Value targets are obtained in two different ways. Firstly, the outcome of the game is used as a (TD(1)) value target for states along the main line of episodes generated by sound self-play. Secondly, value targets are also obtained by bootstrapping: running an instance of GT-CFR from subgames rooted at input queries. In principle, any solver could be used because any subgame rooted at β has well-defined values. Thus, this step acts much like a policy improvement operator via decomposition described in Imperfect Information Search, Decomposition, and Re-Solving. Specifically, the value targets are the final counterfactual values after T iterations of GT-CFR for all the information states within the public state that initiated the search. The specific way that the different value targets are assigned is described by the pseudocode in supplementary text and determined by a hyperparameter noted in Table S2.

Recursive Queries

While the solver is computing targets for a query, it is also generating more queries itself by running GT-CFR. Some of these recursive queries are also added to the buffer for future solving, so that the CVPN can produce reasonable answers for all leaves in a search, not just those on the self-play lines. As a result, at any given time the buffer may include queries generated by search in the main self-play game or by solver-generated queries off the main line. To ensure that the buffer is not dominated by recursive queries, we set the probability of adding a new recursive query to less than 1 (in our experiments, the value is typically 0.1 or 0.2; see Table S2 for the exact values).

Consistency of Training Process

One natural question is whether, or under what circumstances, the training process could ensure convergence to the optimal values? The answer is positive: the training process converges to the optimal values, asymptotically, as $T \to \infty$ and with very large (exponential) memory.

Informally, imagine an oracle function $f(\beta)$ that can simply memorize the values and policy for the particular β similar to a tabular value or policy iteration algorithm except with continuous keys. For any subgame rooted at some β with a depth of 1 (every action leads to terminal states), the values and policies can be computed and stored for β after T iterations of the solver. This can then be applied inductively: since CFR is deterministic, for any subgame on the first iteration of GT-CFR, a finite number of queries will be generated. Each of these queries will be solved using GT-CFR. Eventually, the query will be a specific one that is one step from the terminal state whose values can be computed exactly and stored in $f(\beta)$. As this value was generated in self-play or by a query solver, and CFR is deterministic, it will produce another self-play game with the identical query, except it will load the solved value from $f(\beta)$, and inductively the values will get propagated from the bottom up. Since CFR is deterministic and T is finite, these ensure that the memory requirement is not infinite despite the continuous-valued keys. Practically, the success of the training process will depend on the representational capacity and training efficacy of the function approximation (i.e. neural network architecture).

For a fully-detailed description of the algorithm, including hyperparameter values and specific descriptions of each process described above, see supplementary text.

References

1. A. L. Samuel, Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development* **44**, 206–226 (1959).

2. S. J. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach* (Pearson Education, 2010), third edn.

3. D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, D. Hassabis, Mastering the game of Go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016).

4. M. Campbell, A. J. Hoane, F.-h. Hsu, Deep blue. *Artificial Intelligence* **134**, 57–83 (2002).

5. G. Tesauro, TD-Gammon, a self-teaching backgammon program, achieves master-level play. *Neural Comput.* **6**, 215–219 (1994).

6. D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, D. Hassabis, A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* **632**, 1140–1144 (2018).

7. M. B. Johanson, Robust strategies and counter-strategies: From superhuman to optimal play, Ph.D. thesis, University of Alberta (2016). `http:`

//johanson.ca/publications/theses/2016-johanson-phd-thesis/2016-johanson-phd-thesis.pdf.

8. M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, M. Bowling, Deepstack: Expert-level artificial intelligence in heads-up no-limit poker. *Science* **358** (2017).

9. N. Brown, T. Sandholm, Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. *Science* **360** (2017).

10. N. Brown, T. Sandholm, Superhuman AI for multiplayer poker. *Science* **365**, 885–890 (2019).

11. N. Bard, J. N. Foerster, S. Chandar, N. Burch, M. Lanctot, H. F. Song, E. Parisotto, V. Dumoulin, S. Moitra, E. Hughes, I. Dunning, S. Mourad, H. Larochelle, M. G. Bellemare, M. Bowling, The Hanabi challenge: A new frontier for AI research. *Artificial Intelligence* **280** (2020).

12. A. Lerer, H. Hu, J. Foerster, N. Brown, Improving policies via search in cooperative partially observable games. *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence* (AAAI, 2020).

13. J. Serrino, M. Kleiman-Weiner, D. C. Parkes, J. B. Tenenbaum, Finding friend and foe in multi-agent games. *Proceedings of the Thirty-third Conference on Neural Information Processing Systems* (NeurIPS, 2019).

14. E. Lockhart, N. Burch, N. Bard, S. Borgeaud, T. Eccles, L. Smaira, R. Smith, Human-agent cooperation in bridge bidding. *Proceedings of the Cooperative AI Workshop at 34th Conference on Neural Information Processing Systems* (NeurIPS, 2020).

15. O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, D. Silver, Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* **575**, 350–354 (2019).

16. T. W. Anthony, T. Eccles, A. Tacchetti, J. Kramár, I. M. Gemp, T. C. Hudson, N. Porcel, M. Lanctot, J. Pérolat, R. Everett, S. Singh, T. Graepel, Y. Bachrach, Learning to play no-press Diplomacy with best response policy iteration. *Thirty-third Conference on Neural Information Processing Systems* (NeurIPS, 2020).

17. J. Gray, A. Lerer, A. Bakhtin, N. Brown, Human-level performance in no-press Diplomacy via equilibrium search. *In Proceedings of the International Conference on Learning Representations* (ICLR, 2020).

18. A. Bakhtin, D. Wu, A. Lerer, N. Brown, No-press Diplomacy from scratch. *Proceedings of the Thirty-fourth Conference on Neural Information Processing Systems* (NeurIPS, 2021).

19. N. Brown, T. Sandholm, Safe and nested subgame solving for imperfect-information games. *Proceedings of the 31st Conference on Neural Information Processing Systems* (NIPS, 2017).

20. M. Šustr, M. Schmid, M. Moravčík, N. Burch, M. Lanctot, M. Bowling, Sound search in imperfect information games. *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems* (AAMAS, 2020).

21. V. Kovarík, M. Schmid, N. Burch, M. Bowling, V. Lisý, Rethinking formal models of partially observable multiagent decision making. *Artificial Intelligence* **303**, 103645 (2022).

22. M. Schmid, Search in imperfect information games, Ph.D. thesis, Charles University (2021). `https://arxiv.org/abs/2111.05884`.

23. D. E. Knuth, R. W. Moore, An analysis of alpha-beta pruning. *Artificial Intelligence* **6**, 293–326 (1975).

24. T. A. Marsland, M. Campbell, A survey of enhancements to the alpha-beta algorithm. *ACM Annual Conference*, ACM '81 (Association for Computing Machinery, New York, NY, USA, 1981), p. 109–114.

25. J. Schaeffer, A. Plaat, New advances in alpha-beta searching. *ACM Conference on Computer Science* (Association for Computing Machinery, 1996).

26. S. Gelly, L. Kocsis, M. Schoenauer, M. Sebag, D. Silver, C. Szepesvári, O. Teytaud, The grand challenge of computer Go: Monte Carlo tree search and extensions. *Communications of the ACM* **55**, 106–113 (2012).

27. L. Kocsis, C. Szepesvári, Bandit based Monte-Carlo planning. *In: ECML-06. Number 4212 in LNCS* (Springer, 2006), pp. 282–293.

28. R. Coulom, Efficient selectivity and backup operators in Monte-Carlo tree search. *Computers and Games*, H. J. van den Herik, P. Ciancarini, H. H. L. M. J. Donkers, eds. (Springer Berlin Heidelberg, Berlin, Heidelberg, 2007), pp. 72–83.

29. C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, S. Colton, A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games* **4**, 1-43 (2012).

30. D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, D. Hassabis, Mastering the game of Go without human knowledge. *Nature* **550**, 354–359 (2017).

31. M. Zinkevich, M. Johanson, M. Bowling, C. Piccione, Regret minimization in games with incomplete information. *Advances in Neural Information Processing Systems 20* (NIPS, 2008), pp. 905–912.

32. S. Hart, A. Mas-Colell, A simple adaptive procedure leading to correlated equilibrium. *Econometrica* **68**, 1127–1150 (2000).

33. O. Tammelin, N. Burch, M. Johanson, M. Bowling, Solving heads-up limit Texas Hold'em. *Proceedings of the 24th International Joint Conference on Artificial Intelligence* (IJCAI, 2015).

34. M. Bowling, N. Burch, M. Johanson, O. Tammelin, Heads-up limit Hold'em poker is solved. *Science* **347**, 145–149 (2015).

35. M. Johanson, N. Bard, M. Lanctot, R. Gibson, M. Bowling, Efficient nash equilibrium approximation through Monte Carlo counterfactual regret minimization. *Proceedings of the Eleventh International Conference on Autonomous Agents and Multi-Agent Systems* (AAMAS, 2012).

36. N. Burch, M. Johanson, M. Bowling, Solving imperfect information games using decomposition. *Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence (AAAI)* (AAAI, 2014).

37. N. Brown, A. Bakhtin, A. Lerer, Q. Gong, Combining deep reinforcement learning and search for imperfect-information games. *Thirty-fourth Annual Conference on Neural Information Processing Systems* (NeurIPS, 2020). `https://arxiv.org/abs/2007.13544`.

38. R. Zarick, B. Pellegrino, N. Brown, C. Banister, Unlocking the potential of deep counterfactual value networks. *CoRR* **abs/2007.10442** (2020).

39. N. Brown, T. Sandholm, B. Amos, Depth-limited solving for imperfect-information games. *Proceedings of the Thirty-second Conference on Neural Information Processing Systems* (NeurIPS, 2018).

40. P. I. Cowling, E. J. Powley, D. Whitehouse, Information set Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games* **4**, 120–143 (2012).

41. J. Long, N. R. Sturtevant, M. Buro, T. Furtak, Understanding the success of perfect information Monte Carlo sampling in game tree search. *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, AAAI'10 (AAAI, 2010), p. 134–140.

42. J. Nijssen, M. Winands, Monte-Carlo tree search for the hide-and-seek game scotland yard. *IEEE Transactions on Computational Intelligence and AI in Games* **4**, 282–294 (2012).

43. J. Nijssen, Monte-carlo tree search for multi-player games, Ph.D. thesis, Maastricht University (2013).

44. J. Heinrich, D. Silver, Smooth UCT search in computer poker. *Proceedings of the 24th International Joint Conference on Artificial Intelligence* (IJCAI, 2015).

45. V. Lisý, M. Lanctot, M. Bowling, Online Monte Carlo counterfactual regret minimization for search in imperfect information games. *Proceedings of the Fourteenth International Conference on Autonomous Agents and Multi-Agent Systems* (AAMAS, 2015), pp. 27–36.

46. M. Lanctot, K. Waugh, M. Zinkevich, M. Bowling, Monte Carlo sampling for regret minimization in extensive games. *Advances in Neural Information Processing Systems 22* (NIPS, 2009), pp. 1078–1086.

47. J. Heinrich, M. Lanctot, D. Silver, Fictitious self-play in extensive-form games. *Proceedings of the 32nd International Conference on Machine Learning (ICML 2015)* (2015).

48. M. Lanctot, V. Zambaldi, A. Gruslys, A. Lazaridou, K. Tuyls, J. Perolat, D. Silver, T. Graepel, A unified game-theoretic approach to multiagent reinforcement learning. *Advances in Neural Information Processing Systems* (NeurIPS, 2017).

49. H. Li, K. Hu, S. Zhang, Y. Qi, L. Song, Double neural counterfactual regret minimization. *Proceedings of the Eighth International Conference on Learning Representations* (ICLR, 2019).

50. N. Brown, A. Lerer, S. Gross, T. Sandholm, Deep counterfactual regret minimization. *CoRR* **abs/1811.00164** (2018).

51. E. Steinberger, A. Lerer, N. Brown, DREAM: Deep regret minimization with advantage baselines and model-free learning (2020).

52. S. Srinivasan, M. Lanctot, V. Zambaldi, J. Pérolat, K. Tuyls, R. Munos, M. Bowling, Actor-critic policy optimization in partially observable multiagent environments. *Advances in Neural Information Processing Systems* (NeurIPS, 2018).

53. E. Lockhart, M. Lanctot, J. Pérolat, J.-B. Lespiau, D. Morrill, F. Timbers, K. Tuyls, Computing approximate equilibria in sequential adversarial games by exploitability descent. *Proceedings of the 28th International Joint Conference on Artificial Intelligence* (IJCAI, 2019).

54. D. Hennes, D. Morrill, S. Omidshafiei, R. Munos, J. Perolat, M. Lanctot, A. Gruslys, J.-B. Lespiau, P. Parmas, E. Duenez-Guzman, K. Tuyls, Neural replicator dynamics. *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems* (AAMAS, 2020).

55. A. Gruslys, M. Lanctot, R. Munos, F. Timbers, M. Schmid, J. Perolat, D. Morrill, V. Zambaldi, J.-B. Lespiau, J. Schultz, M. G. Azar, M. Bowling, K. Tuyls, The advantage regret-matching actor-critic (2020).

56. J. Perolat, R. Munos, J.-B. Lespiau, S. Omidshafiei, M. Rowland, P. Ortega, N. Burch, T. Anthony, D. Balduzzi, B. D. Vylder, G. Piliouras, M. Lanctot, K. Tuyls, From Poincaré recurrence to convergence in imperfect information games: Finding equilibrium via regularization. *Proceedings of the The Thirty-eighth International Conference on Machine Learning (ICML)* (2021).

57. S. McAleer, J. Lanier, P. Baldi, R. Fox, Xdo: A double oracle algorithm for extensive-form games. *Proceedings of the Thirty-fifth Conference on Neural Information Processing Systems* (NeurIPS, 2021).

58. X. Feng, O. Slumbers, Z. Wan, B. Liu, S. M. McAleer, Y. Wen, J. Wang, Y. Yang, Neural auto-curricula in two-player zero-sum games. *Proceedings of the Thirty-fifth Conference on Neural Information Processing Systems* (NeurIPS, 2021).

59. J. Perolat, B. D. Vylder, D. Hennes, E. Tarassov, F. Strub, V. de Boer, P. Muller, J. T. Connor, N. Burch, T. Anthony, S. McAleer, R. Elie, S. H. Cen, Z. Wang, A. Gruslys, A. Malysheva, M. Khan, S. Ozair, F. Timbers, T. Pohlen, T. Eccles, M. Rowland, M. Lanctot, J.-B. Lespiau, B. Piot, S. Omidshafiei, E. Lockhart, L. Sifre, N. Beauguerlange, R. Munos, D. Silver, S. Singh, D. Hassabis, K. Tuyls, Mastering the game of Stratego with model-free multiagent reinforcement learning. *Science* **378**, 990-996 (2022).

60. T. Dash, S. N. Dambekodi, P. N. Reddy, A. Abraham, Adversarial neural networks for playing hide-and-search board game Scotland Yard. *Neural Computing and Applications* **32**, 3149–3164 (2018).

61. J. N. Foerster, F. Song, E. Hughes, N. Burch, I. Dunning, S. Whiteson, M. Botvinick, M. Bowling, Bayesian action decoder for deep multi-agent reinforcement learning (2019).

62. S. Sokota, E. Lockhart, F. Timbers, E. Davoodi, R. D'Orazio, N. Burch, M. Schmid, M. Bowling, M. Lanctot, Solving common-payoff games with approximate policy iteration. *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence* (AAAI, 2021).

63. D. Strouse, K. R. McKee, M. Botvinick, E. Hughes, R. Everett, Collaborating with humans without human data. *Proceedings of the Thirty-fifth Conference on Neural Information Processing Systems* (NeurIPS, 2021).

64. A. Bakhtin, N. Brown, E. Dinan, G. Farina, C. Flaherty, D. Fried, A. Goff, J. Gray, H. Hu, A. P. Jacob, M. Komeili, K. Konath, M. Kwon, A. Lerer, M. Lewis, A. H. Miller, S. Mitts, A. Renduchintala, S. Roller, D. Rowe, W. Shi, J. Spisak, A. Wei, D. Wu, H. Zhang, M. Zijlstra, Human-level play in the game of Diplomacy by combining language models with strategic reasoning. *Science* **378**, 1067-1074 (2022).

65. T. S. D. Team, Stockfish: Open source chess engine (2021). `https://stockfishchess.org/`.

66. M. Johanson, Measuring the size of large no-limit poker games (2013).

67. Game of the year 1983: Scotland Yard, `https://www.spiel-des-jahres.de/spiel-des-jahres-1983-scotland-yard/`.

68. F. Southey, M. Bowling, B. Larson, C. Piccione, N. Burch, D. Billings, C. Rayner, Bayes' bluff: Opponent modelling in poker. *Proceedings of the Twenty-First Conference on Uncertaintyin Artificial Intelligence (UAI)* (2005), pp. 550–558.

69. A. Elo, *The Rating of Chessplayers, Past and Present* (Arco Pub., 1986), second edn.

70. T. G. D. Team, Gnugo (2009). `https://www.gnu.org/software/gnugo/`.

71. P. Baudis, J. loup Gailly, Lemonsqueeze, Pachi: Software for the board game of go / weiqi / baduk (2016). `https://pachi.or.cz/`.

72. E. Jackson, Slumbot NL: Solving large games with counterfactual regret minimization using sampling and distributed processing. *Proceedings of the Computer Poker and Imperfect Information: Papers from the AAAI 2013 Workshop* (2013). `https://github.com/ericgjackson/slumbot2019`.

73. E. Jackson, Slumbot github repository. `https://github.com/ericgjackson/slumbot2017`.

74. N. Burch, M. Schmid, M. Moravčík, M. Bowling, AIVAT: A new variance reduction technique for agent evaluation in imperfect information games (2017).

75. V. Lisý, M. Bowling, Eqilibrium approximation quality of current no-limit poker bots. *Workshops at the Thirty-First AAAI Conference on Artificial Intelligence* (2017).

76. R. B. Segal, On the scalability of parallel UCT. *CG'10: Proceedings of the 7th international conference on Computers and games* (2010), pp. 36–47.

77. P. J. Huber, Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics* **35**, 73 – 101 (1964).

78. N. Brown, T. Sandholm, Strategy-based warm starting for regret minimization in games. *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence* (AAAI, 2016), pp. 432–438.

79. M. Lanctot, E. Lockhart, J.-B. Lespiau, V. Zambaldi, S. Upadhyay, J. Pérolat, S. Srinivasan, F. Timbers, K. Tuyls, S. Omidshafiei, D. Hennes, D. Morrill, P. Muller, T. Ewalds, R. Faulkner, J. Kramár, B. D. Vylder, B. Saeta, J. Bradbury, D. Ding, S. Borgeaud, M. Lai, J. Schrittwieser, T. Anthony, E. Hughes, I. Danihelka, J. Ryan-Davis, OpenSpiel: A framework for reinforcement learning in games. *CoRR* **abs/1908.09453** (2019).

80. M. Šustr, V. Kovařík, V. Lisỳ, Monte Carlo continual resolving for online strategy computation in imperfect information games. *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems* (2019), pp. 224–232.

Acknowledgements

We thank several people for their help and feedback: Ed Lockhart, Michael Johanson, Adam White, Julian Schrittwieser, Thomas Hubert, Michal Sustr, Leslie Acker, Morgan Redshaw, Dustin Morrill, Trevor Davis, Stephen McAleer, Sanah Choudry, and Shayna Bowling.

We would like to extend a special thanks to Mark Winands and Pim Nijssen for providing the code for and helping us configure their Scotland Yard agent, and special thanks to Eric Jackson for providing the code for and assistance with his poker agent.

Author Contributions: Conceptualization: NBa, MB, NBu, MM, MS, KW; Methodology: NBu, MM, MS, KW; Software: NBa, NBu, JD, ZH, RK, ML, MM, MS, FT, KW; Validation: NBa, JD, RK, KW; Formal analysis: NBu; Investigation: NBa, NBu, JD, ED, RK, ML, MM, MS, FT, KW; Writing — original draft: NBa, MB, NBu, RK, ML, MM, MS; Writing — review & editing: NBa, MB, NBu, ML; Visualization: NBa, JD, RK; Supervision: NBa, MB, MS; Project administration: NBa, MB, AC, JD.

Competing Interests: The authors declare they have no competing interest.

Data and Materials Availability: All data needed to evaluate the conclusions of the paper are present in the paper and/or the Supplementary Materials.

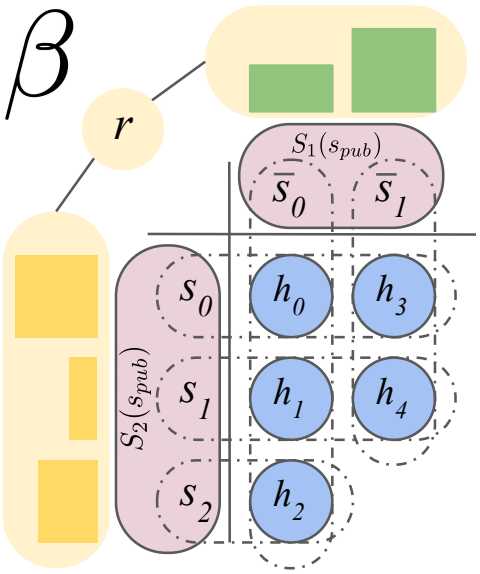


Figure 1: An example structure of public belief state $\beta = (s_{\textbf{pub}}, r)$. s_{pub} translates to two sets of information states, one for player 1, $\mathcal{S}_1(s_{\text{pub}}) = \{\bar{s}_0, \bar{s}_1\}$, and one for player 2, $\mathcal{S}_2(s_{\text{pub}}) = \{s_0, s_1, s_2\}$. Each information state includes different partitions of possible histories. Finally r contains reach probabilities for information states for both players.

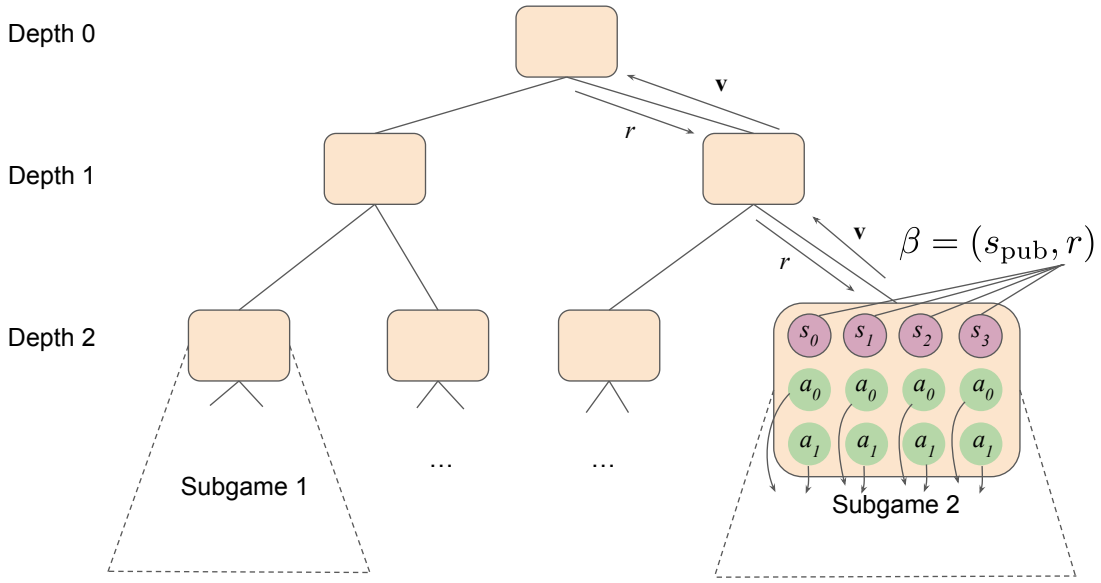
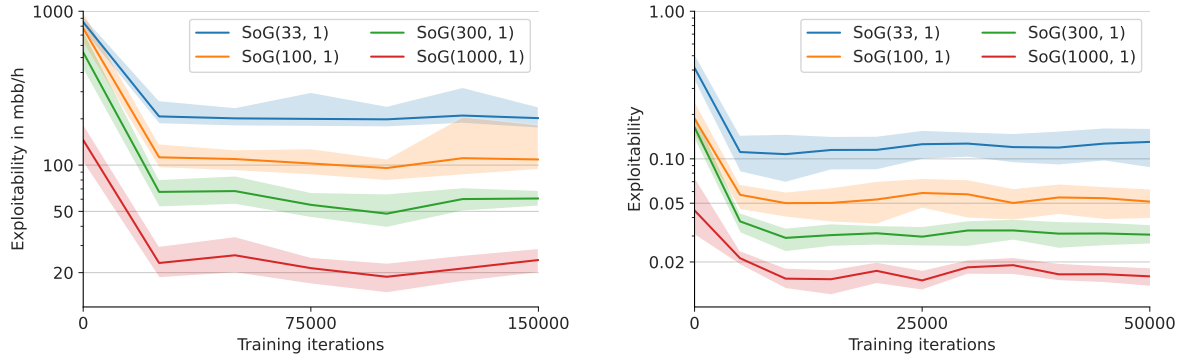


Figure 2: An example of depth-limited CFR solving using decomposition in a game with two specific subgames shown. Standard CFR would require traversing all the subgames. Depth-limited CFR decomposes the solve into running down to depth $d = 2$ and using $\mathbf{v} = \mathbf{v}_{\boldsymbol{\theta}}(\beta)$ to represent the second subgame's values. On the downward pass, ranges r are formed from policy reach probabilities. Values are passed back up to tabulate accumulating regrets. Re-solving a subgame would require construction of an auxiliary game (*36*) (not shown).



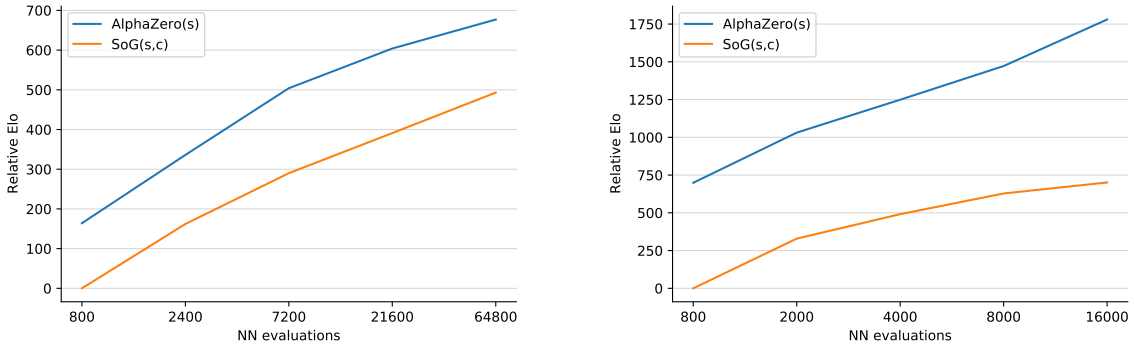
(**A**) Leduc poker

(**B**) Scotland Yard (glasses map)

Figure 3: Exploitability of SoG as a function of the number of training steps under different number of simulations of GT-CFR. For both (A) Leduc poker and (B) Scotland Yard (glasses map), each line corresponds to a different evaluation condition, e.g. SoG(s, c) used at evaluation time. The ribbon shows minimum and maximum exploitability out of 50 seeded runs for each setup. The units of the y-axis in Leduc poker are milli big blinds per hand (mbb/h), which corresponds to one thousandth of a chip in Leduc. In Scotland Yard the reward is either -1 (loss) or +1 (win). All networks were trained using a single training run of SoG$(100, 1)$, and the x-values correspond to a network trained for the corresponding number of steps.

Chess Agents	Rel. Elo	Go Agents	Rel. Elo
AlphaZero(sims=60k)	+592	AlphaZero(s=16k, t=800k)	+3139
Stockfish(threads=16, time=4s)	+530	AlphaZero(s=8k, t=800k)	+2875
AlphaZero(sims=8k)	+455	**SoG(s=16k, c=10)**	**+1970**
SoG(s=60k, c=10)	**+420**	**SoG(s=8k, c=10)**	**+1902**
Stockfish(threads=4, time=1s)	+382	Pachi(s=100k)	+869
SoG(s=8k, c=10)	**+268**	Pachi(s=10k)	+231
Stockfish(threads=1, time=0.1s)	0	GnuGo(l=10)	0

Table 1: Relative Elo of different agents in chess (left), and Go (right). Each agent played 200 matches (100 as white and 100 as black) against every other agent in the tournament. For chess, Elo of Stockfish with a single thread and 100ms thinking time was set to be 0. For Go, Elo of GnuGo was set to be 0. The other values are relative to those. AlphaZero(s=16k, t=800k) refers to 16000 search simulations. For full results, see Tables S3 and S4.



(**A**) Chess (**B**) Go

Figure 4: Scalability of SoG with increasing number of neural network evaluations compared to AlphaZero measured on relative Elo scale. The x-axis corresponds to the number of simulations in AlphaZero and s in SoG(s, c). Elo of SoG$(s = 800, c)$ was set to be 0. In chess (A), $c = 10$ for all runs, with varying $s \in \{800, 2400, 7200, 21600, 64800\}$. In Go (B), we graph SoG using $(s, c) \in \{(800, 1), (2000, 10), (4000, 10), (8000, 10), (16000, 16).\}$

Agent Name	**Slumbot**	**LBR** (*75*)
Slumbot (2016)	-	-522 $\pm$ 50
ARMAC (*55*)	-	-460 $\pm$ 260
DeepStack (*8*)	-	428 $\pm$ 87
Modicum (*39*)	11 $\pm$ 5	-
ReBeL (*37*)	45 $\pm$ 5	-
Supremus (*38*)	176 $\pm$ 44	951 $\pm$ 96
SoG$(10, 0.01)$	7 $\pm$ 3	434 $\pm$ 9

Table 2: Head-to-head results showing expected winnings (mbb/h) of SoG and other recently published agents against Slumbot and LBR. The LBR agent use either fold or call (FC) actions in the all four rounds. The $\pm$ shows one standard error. LBR results for Slumbot are from (*75*). The other results are from the papers describing the agents.

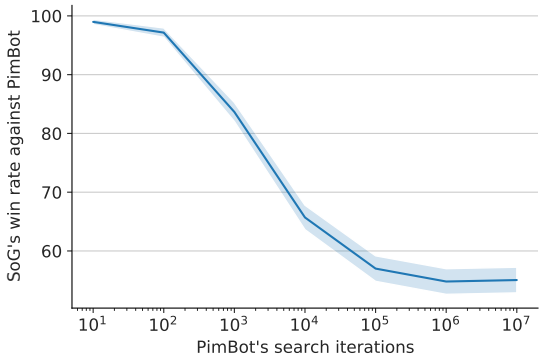


Figure 5: Win rate of SoG$(400, 1)$ against PimBot with varying simulations. 2000 matches were played for each data point, with roles swapped for half of the matches. Note that the x-axis has logarithmic scale. The ribbon shows 95% confidence interval.

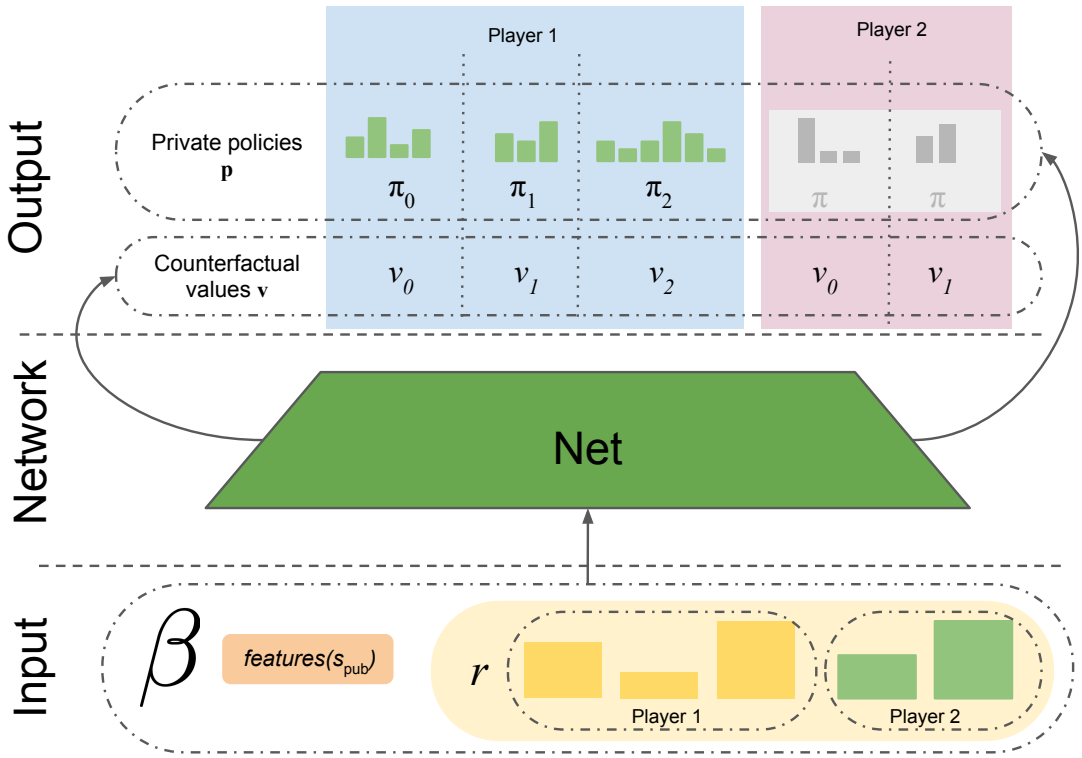


Figure 6: A counterfactual value-and-policy network (CVPN). Each query, β, to the network includes beliefs r and an encoding of s_{pub} to get the counterfactual values $\mathbf{v}$ for both players and policies $\mathbf{p}$ for the acting player in each information state $s_i \in s_{\text{pub}}(h)$, producing outputs $f_{\boldsymbol{\theta}}$. Since players may have different actions spaces (as in e.g. Scotland Yard) there are two sets of policy outputs: one for each player, and $\mathbf{p}$ refers to the one for the acting player at s_{pub} only (depicted as player 1 in this diagram by greying out player 2's policy output).

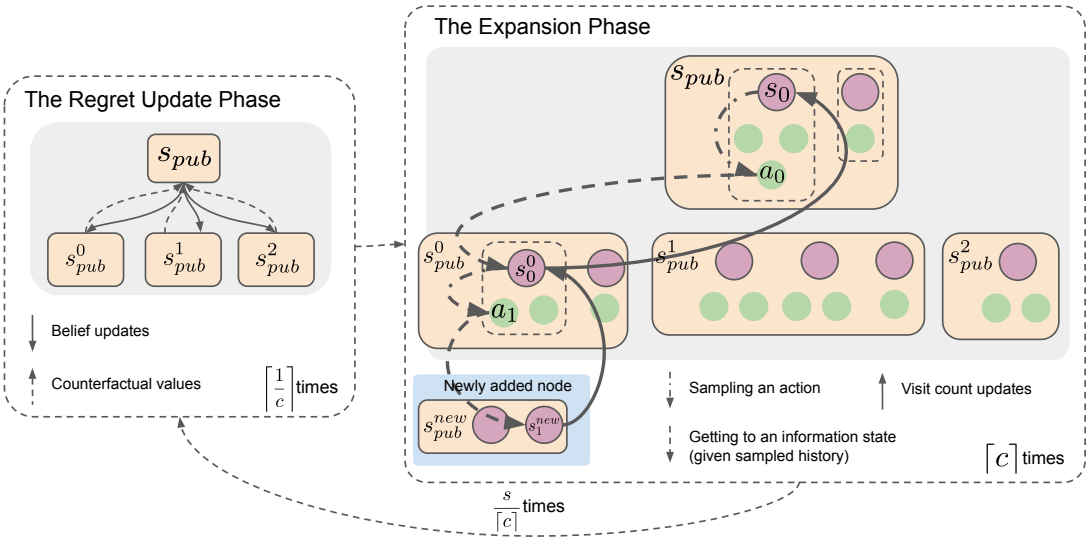


Figure 7: Overview of the phases in one iteration of Growing-Tree CFR. The regret update phase propagates beliefs down the tree, obtains counterfactual values from the CPVN at leaf nodes (or from the environment at terminals), and passes back counterfactual values to apply the CFR update. The expansion phase simulates a trajectory from the root to a leaf, adding public states to the tree. In this case the trajectory starts in the public belief state s_{pub} by sampling the information state s_0. After that the sampled action a_0 leads to the information state s^0_0 in public state s^0_{pub}, and finally the action a_1 leads to a new public state that is added to the tree.

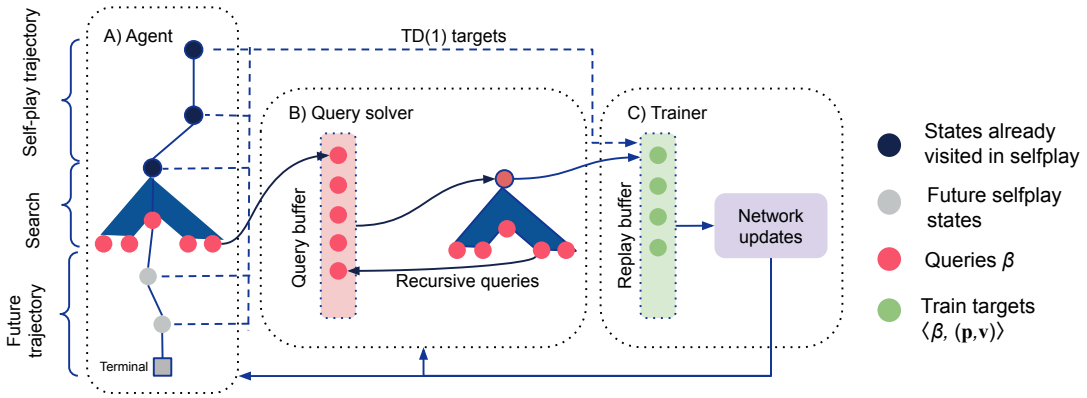


Figure 8: SoG Training Process. Actors collect data via sound self-play and trainers run separately over a distributed network. (A) Each search produces a number of CVPN queries with input β. (B) Queries are added to a query buffer and subsequently solved by a solver that studies the situation more closely via another invocation of GT-CFR. During solving, new recursive queries might be added back to the query buffer; separately the network is (C) trained on minibatches sampled from the replay buffer to predict values and policy targets computed by the solver.

ScienceAdvances
AAAS

Supplementary Materials for

Student of Games: A unified learning algorithm for both perfect and imperfect information games

Martin Schmid *et al.*

*Corresponding author. Email: mbowling@ualberta.ca

This PDF file includes:

Supplementary Text
Figs. S1 to S4
Tables S1 to S6
References (77 to 80)

Supplementary Text

Student of Games Algorithm Details

Network Architecture and Optimization

Table S1 lists neural network architectures and input features used for each game. For chess and Go we use exactly the same architecture and inputs as used by AlphaZero (*6*). In poker and Scotland Yard we process concatenated belief and public state features by a MLP with ReLU activations.

The counterfactual value head is optimized by Huber loss (*77*), while policy for each information state i is optimized by KL divergence:

$$l(\mathbf{v}, \mathbf{p}, \mathbf{v}_{target}, \mathbf{p}_{target}) = w_v * l_{huber}(\mathbf{v}, \mathbf{v}_{target}) + w_p * \sum_i l_{KL}(\pi^i, \pi^i_{target})$$

where each head is weighted with the corresponding weight w_v and w_p. During training we smoothly decay the learning rate by a factor of d every T_{decay} steps. Formally learning rate α at training step t is defined as:

$$\alpha_t = \alpha_{init} * d^{t/T_{decay}}$$

When using the policy head's prediction as prior in PUCT formula the logits are processed with softmax with temperature T_{prior}. This can decrease weight of the prior in some games and encourage more exploration in the search phase.

Pseudocode

Here we provide pseudocode for the most important parts of the SoG algorithm. Algorithm 1 specifies GT-CFR, the core of SoG's sound game-theoretic search that scales to large perfect information games introduced in Search via Growing-Tree CFR. Algorithm 2 presents how GT-CFR is used during self-play that generates training examples for the neural network, previously covered in Training Process. Hyperparameters used in self-play are specified in Table S2.

When SoG plays against an opponent, the search tree is rebuilt also for the opponent's actions (as discussed in Modified Continual Re-solving). This way, SoG reasons about the opponent's behavior since it directly influences the belief distribution for the current state β where SoG is to act.

Note that unlike AlphaZero, SoG currently starts its search procedure from scratch. That is, the previous computation only provides invariants for the next resolving step. AlphaZero rather warm-starts the MCTS process by initializing values and visit counts from the previous search. For SoG, this would also require warm-starting CFR. While possible (*78*), there is no warm-starting in the current implementation of SoG.

Algorithm 1 Growing Tree CFR. Note that GT-CFR is logging all neural net queries it does since they might be used later in training.

procedure GT-CFR($\mathcal{L}^0, \beta, s, c$)
▷ $\mathcal{L}^0$ — a tree including β built as described in Modified Continual Re-solving.
▷ β — a public belief state under which the new nodes will be added.
▷ s, c — total number of expansion simulations and number of simulations per CFR update.
for $i \in \{0, 1, \cdots, \frac{s}{\lceil c \rceil} - 1\}$ **do**
CFR($\mathcal{L}^i, \left\lceil \frac{1}{c} \right\rceil$) ▷ Store average policy and counterfactual values in the tree.
$\mathcal{L}^{i+1} \leftarrow$ GROW($\mathcal{L}^i$)
end for
▷ Return counterfactual values and average policy from CFR and all NN calls.
return $\mathbf{v}, \mathbf{p}, nn_queries$
end procedure

procedure GROW($\mathcal{L}, \beta$)
for $i \in \{0, 1, \cdots, \lceil c \rceil - 1\}$ **do**
$path \leftarrow$ SAMPLEPATHDOWNTHETREE($\mathcal{L}, \beta$) ▷ The path starts at β.
ADDTOPKCHILDREN($\mathcal{L}, path, k$)
UPDATEVISITCOUNTSUP($\mathcal{L}, path$)
end for
return $\mathcal{L}$
end procedure

Algorithm 2 Sound Self-play

procedure SELFPLAY
  Get initial history state $w \leftarrow w^{INIT}$ and corresponding public state β
  ▷ Decide whether the game has to be played till the end.
  $do_not_resign \leftarrow$ coin flip with probability p_{no_resign}
  while w is not terminal AND played less than $moves_{max}$ **do**
    if chance acts in w **then**
      $a \leftarrow$ uniform random action.
    else
      ▷ SoG acts for all non-chance players.
      $v_w, \pi_w^{controller} \leftarrow$ SOGSELFPLAYCONTROLLER(w)
      if $v_w < resign_threshold$ AND $not(do_not_resign)$ **then**
        ▷ Don't waste compute on already decided game.
        return
      end if
      ▷ Mix controller's policy with uniform prior to encourage exploration.
      $\pi_w^{selfplay} \leftarrow (1-\epsilon) \cdot \pi_w^{controller} + \epsilon \cdot \pi^{uniform}$
      if moves played $< moves_{greedy_after}$ **then**
        $a \leftarrow$ sample action from $\pi_w^{selfplay}$
      else
        $a \leftarrow \arg\max \pi_w^{selfplay}$
      end if
    end if
    $w \leftarrow$ apply action a on state w
  end while
  ▷ Sampling states with TD(1) targets.
  for each belief state $\beta \in$ played trajectory tr **do**
    if uniform random sample from unit interval $< p_{td1}$ **then**
      $\mathbf{v} \leftarrow$ outcome of tr assigned to information state visited in β
      $\mathbf{p} \leftarrow$ policy used in β
      replay_buffer.append($\langle \beta, (\mathbf{v}, \mathbf{p}) \rangle$)
    end if
  end for
end procedure

procedure SOGSELFPLAYCONTROLLER(w)
  $\beta \leftarrow$ public state including w
  $\mathcal{L} \leftarrow$ the tree including β built as described in Modified Continual Re-solving.
  $\mathbf{v}, \mathbf{p} \leftarrow$ TRAINING-GT-CFR($\mathcal{L}$)
  return $\mathbf{v}(w), \mathbf{p}(w)$
end procedure

procedure TRAINING-GT-CFR($\mathcal{L}$)
  $\mathbf{v}, \mathbf{p}, nn_queries \leftarrow$ GT-CFR($\mathcal{L}$)
  queries $\leftarrow$ Pick on average q_{search} neural net queries β from $nn_queries$.
  queries_to_solve.extend(queries)
  return $\mathbf{v}, \mathbf{p}$
end procedure

procedure QUERYSOLVER
  for $\beta \leftarrow$ queries_to_solve.pop() **do**
    $\mathbf{v}, \mathbf{p}, nn_queries \leftarrow$ GT-CFR(β)
    ▷ Send the example to the trainer.
    replay_buffer.append($\langle \beta, (\mathbf{v}, \mathbf{p}) \rangle$)
    ▷ Create recursive queries.
    queries $\leftarrow$ Pick on average $q_{recursive}$ neural net queries β from $nn_queries$.
    queries_to_solve.extend(queries)
  end for
end procedure

Implementation

SoG is implemented as a distributed system with decoupled actor and trainer jobs. Each actor runs several parallel games and the neural network evaluations are batched for better accelerator utilization. The networks were implemented using TensorFlow.

Poker Betting Abstraction

There are up to 20000 possible actions in no-limit Texas hold'em. To make the problem easier, AI agents are typically allowed to use only a small subset of these (*8, 37–39*). This process of selecting a set of allowed actions for a given poker state is called betting abstraction. Even using betting abstraction the players are able to maintain strong performance in the full game (*8, 37, 38*). Moreover, the local best response evaluation (*75*) suggests that there is not an easy exploit for such simplification as long as the agent is able to see full opponent actions (*8*).

We use a betting abstraction in the Student of Games to speed up the training and simplify the learning task. Our agent's action set was limited to just 3 actions: fold (give up), check/call (match the current wager) and bet/raise (add chips to the pot). To improve generalization we used stochastic betting size similarly to ReBeL (*37*). The single allowed bet/raise size is randomly uniformly selected at the start of each poker hand from the interval $\langle 0.5, 1.0 \rangle * pot_size$. This amount is anecdotally similar to one used by human players and had good performance in our experiments. The same random selection was used in both training and evaluation.

As in (*37*), we have also randomly varied the stack size (number of chips available to the players) at the start of the each round during the training. This number stays fixed during evaluation.

Evaluation Details and Additional Experimental Results

Description of Leduc poker

Leduc is a simplified poker game with two rounds and a 6-card deck in two suits. Each player initially antes a single chip to play and obtains a single private card and there are three actions: fold, call and raise. There is a fixed bet amount of 2 chips in the first round and 4 chips in the second round, and a limit of two raises per round. After the first round, a single public card is revealed. A pair is the best hand, otherwise hands are ordered by their high card (suit is irrelevant). A player's reward is their gain or loss in chips after the game.

Reinforcement Learning and Search in Imperfect Information Games

In this section, we provide some experimental results showing that common RL and widely-used search algorithms can produce highly exploitable strategies, even in small imperfect information games where exploitability is computable exactly. In particular, we show how ex-

ploitable Information Set Monte Carlo Tree Search is in Leduc poker, as well as three standard RL algorithms (DQN, A2C and tabular Q-learning) in both Kuhn poker and Leduc poker using OpenSpiel (*79*). Results are presented in milli big blinds per hand (mbb/h), which corresponds to one thousandth of a chip for both games.

Information Set Monte Carlo Tree Search

Information Set Monte Carlo Tree Search (IS-MCTS) is a search method that, at the start of each simulation, first samples a world state– consistent with the player's information state– and uses it for the simulation (*40*). Reward and visit count statistics are aggregated over information states so that players base their decisions only on their information states rather than on private information inaccessible to them.

Table S5 shows the exploitability of a policy obtained by running separate independent IS-MCTS searches from each information state in the game, over various parameter values. The lowest exploitability of IS-MCTS we found among this sweep was **465 mbb/h**.

Standard RL algorithms in Imperfect Information Games

As imperfect information games generally need stochastic policies to achieve an optimal strategy, one might wonder how exploitable standard RL algorithms are in this class of games. To test this, we trained three standard RL agents: DQN, policy gradient (A2C) and tabular Q-learning. We used MLP neural networks in DQN and A2C agents. Table S6 shows the hyper parameters we swept over to train these RL agents.

In Kuhn poker, the best performing A2C agent converges to exploitability of 52 mbb/h, and tabular Q-learning and DQN agents converge to around 250 mbb/h. Similarly, in Leduc poker, the best performing A2C agent converges to exploitability of 78 mbb/h, tabular Q-learning and DQN agents converge to about 1300 mbb/h and 900 mbb/h respectively. Figure S4 shows the exploitability of RL agents in Kuhn poker and Leduc poker.

Proofs of Theorems

There are three substantive differences between the SoG algorithm and DeepStack. First, SoG uses a growing search tree, rather than using a fixed limited-lookahead tree. Second, the SoG search tree may depend on the observed chance events. Finally, SoG uses a continuous self-play training loop operating throughout the entire game, rather than the stratified bottom-up training process used by DeepStack. We address each of these differences below, in turn, after considering how to describe an approximate value function for search in imperfect information games.

Value Functions for Subgames

Like DeepStack, the SoG algorithm uses a value function, so the quality of its play depends on the quality of the value function. We will describe a value function in terms of its distance to a strategy with low regret. We start with some value and regret definitions that are better suited to subgames.

Consider some policy profile π which is a tuple containing a strategy for each player, public tree subgame S rooted at public state s_{pub} with player ranges $B_i[s_i \in \mathcal{S}_i(s_{\text{pub}})] \coloneqq P_i(s_i|\pi)$. First, note that we can re-write counterfactual value v so that it depends only on B and π restricted to S, with no further dependence on π. Let s_i be a Player i information state in $\mathcal{S}_i(s_{\text{pub}})$, and q be the opponent of Player i, then:

$$\begin{aligned} v^{\mathbf{B},\pi^S}(s_i) &\coloneqq \sum_{h \in I(s_i)} \sum_{z \sqsupset h} B_q[s_q(h)] P_c(h) P(z|h,\pi^S) u_i(z) \\ &= \sum_{h \in I(s_i)} \sum_{z \sqsupset h} P_{-i}(h|\pi) P(z|h,\pi) u_i(z) = v^{\pi}(s_i) \end{aligned}$$

We can write several quantities in terms of the best-response value at information state s_i:

$$BV^{B,\pi^S}(s_i) \coloneqq \max_{\pi_i^*} v^{B,\pi^S \leftarrow \pi_i^*}(s_i)$$

where $\pi \leftarrow \pi'$ is the policy profile constructed by replacing action probabilities in π with those in π'. The value function is a substitute for an entire subgame policy profile, so the regret we are interested in is player i's full counterfactual regret (*31*) at s_i, which considers all possible strategies within subgame S:

$$R^{\text{full}}_{s_i}(B,\pi^S) \coloneqq BV^{B,\pi^S}(s_i) - v^{B,\pi^S}(s_i)$$

With these definitions in hand, we can now consider the quality of a value function f in terms of a regret bound ϵ and value error ξ. Recall that f maps ranges B and public state s_{pub} to approximate counterfactual values $\tilde{v}(s_i)$ for each player i.

First, we consider versions of the regret bound and value error which are parameterised by a strategy π. There is some associated bound $\epsilon(\pi)$ on the sum of regrets across all information states at any subgame, valid for both players.

$$\epsilon(\pi) \coloneqq \max_B \max_{s_{\text{pub}}} \max_i \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} R^{\text{full}}_{s_i}(B,\pi)$$

There is also some bound $\xi_f(\pi)$ on the distance between $f(s_{\text{pub}}, B)$ and the best-response values to π.

$$\xi_f(\pi) \coloneqq \max_B \max_{s_{\text{pub}}} \max_i \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} |f(s_{\text{pub}}, B)[s_i] - v^{B,\pi}(s_i)|$$

We then say that f has ϵ, ξ quality bounds if there exists some strategy π such that $\epsilon(\pi) \leq \epsilon$ and $\xi_f(\pi) \leq \xi$. As desired, if both ϵ and ξ are low then $f(s_{\text{pub}}, B)$ is a good approximation of the best-response values to a low-regret strategy, for a subgame rooted at s_{pub} with initial beliefs B.

The DeepStack algorithm (*8*) used a similar error metric for value functions, but only considered zero-regret strategies. We introduce a more complicated error measure because the space of values corresponding to low-regret strategies may be much larger than the space of values corresponding to no-regret strategies. For example, consider the public subgame of a matching pennies game after the first player acts with the policy 0.501 heads, 0.499 tails. There are two first-player information states, from playing either heads or tails, with an empty first-player strategy as there are no further first player actions. Let us assume a value function f is returning the values $[0\ 0]$ for these two information states. How good is f, assuming we restrict our attention to this one subgame?

The unique zero-regret strategy for the second player is to play tails 100% of the time, resulting in first player counterfactual values of -1 for playing heads and 1 for playing tails. The error metric based on zero-regret strategies is therefore measuring $|f([0.501\ 0.499]) - [-1\ 1]|_1$, so that the DeepStack metric states that f has an error of 2. However, $[0\ 0]$ seems like a very reasonable choice: these are exactly the first player counterfactual values when the second player has a strategy of 0.5 heads, 0.5 tails, which has a regret of only 0.002 in this subgame. Rather than saying f is a poor quality value function with an error of 2 in a game with utilities in $[-1, 1]$, we can now say f is a great $0.002, 0$ value function which exactly describes a low-regret strategy.

The new quality metric also addresses an issue the old DeepStack metric had with discontinuities in the underlying 0-regret value functions. This means that the space of functions with a low DeepStack error metric may not be well suited for learning from data. Continuing with the previous example, if we shifted B slightly to be 0.499 heads and 0.501 tails for the first player, the unique 0-regret strategy in the subgame flips to playing tails 0% of the time, while the uniform random strategy is still a low-regret strategy for this subgame. In this example, a function can only have a low error with the DeepStack metric if it accurately predicts the values everywhere around the discontinuity at 0.5 heads and 0.5 tails, whereas the new metric can avoid this discontinuity by picking an $\epsilon > 0$. More generally, for any constant c, the objective $\epsilon + c\xi$ is a continuous function in B, making it a potentially more attractive learning target than the discontinuous function defined by exact Nash equilibrium values, and which matches a learning procedure based on approximately solving example subgames.

Growing Trees

One major step in showing soundness of the SoG algorithm is demonstrating that Growing Tree CFR (GT-CFR) can approximately solve games. As a quick recap, GT-CFR is a variant of the CFR algorithm (*31*) that uses limited lookahead and a value function, storing values within a tree that grows over time, in a fashion similar to UCT (*27*). We use this algorithm as a

component to solve the problems that the SoG algorithm sets up. At every non-terminal public leaf state s_{pub} of the lookahead tree, GT-CFR uses estimated counterfactual values $\tilde{v}$, generated from a value function $f(s_{\text{pub}}, B)$ with player ranges B induced by Bayes' rule at s_{pub} for the current policy profile π.

Like DeepStack, SoG has two steps which involve solving subgames of the original game. One of the steps is the re-solving step used to play through a game, where we solve a modified subgame based on constraints on opponent values and beliefs about our possible private information, in order to get our policy and new opponent values. The other step is only in the training loop, where we are solving a subgame with fixed beliefs for both players, in order to get values for both players. While the (sub)games for these two cases are slightly different, they are both well-formed games and we can find an approximate Nash equilibrium using GT-CFR.

When running GT-CFR, even though a policy is explicitly defined only at information states in the lookahead tree $\mathcal{L}$, at each iteration t there is implicitly some complete policy profile π^t. For any information state s in $\mathcal{L}$ which is not a leaf, $\pi^t(s)$ is explicitly defined by the regret-matching policy. For all other s – either a leaf of $\mathcal{L}$ or outside of the lookahead tree – $\pi^t(s)$ is defined by the ϵ-regret subgame policy profile $\pi^{*,S}$ associated with the value function's ϵ, ξ quality bounds. Note that this π^t only exists as a concept which is useful for theoretical analysis: GT-CFR does not have access to the probabilities outside of its lookahead tree, only a noisy estimate of the associated counterfactual values provided by the value function.

Lemma 1. *Let* $\mathbf{p}$ *and* $\mathbf{q}$ *be vectors in* $[0,1]^n$*, and* $\mathbf{v}$ *and* $\mathbf{w}$ *be vectors in* $\mathbb{R}^n$ *such that* $\mathbf{v}[\mathrm{i}] > \mathbf{w}[\mathrm{i}]$ *for all* i*. Then* $\mathbf{p}\cdot\mathbf{v} - \mathbf{q}\cdot\mathbf{w} \leq \mathbf{1}\cdot(\mathbf{v}-\mathbf{w}) + \mathbf{p}\cdot\mathbf{w} - \mathbf{q}\cdot\mathbf{w}$

Proof.

$$\begin{aligned}\mathbf{p}\cdot\mathbf{v} - \mathbf{q}\cdot\mathbf{w} &= \mathbf{p}\cdot\mathbf{v} - \mathbf{p}\cdot\mathbf{w} + \mathbf{p}\cdot\mathbf{w} - \mathbf{q}\cdot\mathbf{w} \\ &= \mathbf{p}\cdot(\mathbf{v}-\mathbf{w}) + \mathbf{p}\cdot\mathbf{w} - \mathbf{q}\cdot\mathbf{w} \\ &\leq \mathbf{1}\cdot(\mathbf{v}-\mathbf{w}) + \mathbf{p}\cdot\mathbf{w} - \mathbf{q}\cdot\mathbf{w}\end{aligned}$$

□

Lemma 2. *Let* $\mathbf{p}$ *and* $\mathbf{q}$ *be vectors in* $[0,1]^n$*, and* $\mathbf{v}$ *and* $\mathbf{w}$ *be vectors in* $\mathbb{R}^n$ *such that* $\sum_{i=1}^{n} |\mathbf{v}[\mathrm{i}] - \mathbf{w}[\mathrm{i}]| \leq \xi$*. Then* $(\mathbf{p}-\mathbf{q})\cdot\mathbf{v} \leq \xi + (\mathbf{p}-\mathbf{q})\cdot\mathbf{w}$.

Proof.

$$\begin{aligned}(\mathbf{p}-\mathbf{q})\cdot\mathbf{v} &= (\mathbf{p}-\mathbf{q})\cdot(\mathbf{v}-\mathbf{w}) + (\mathbf{p}-\mathbf{q})\cdot\mathbf{w} \\ &\leq \sum_{i=1}^{n} |(\mathbf{p}[\mathrm{i}] - \mathbf{q}[\mathrm{i}])(\mathbf{v}[\mathrm{i}] - \mathbf{w}[\mathrm{i}])| + (\mathbf{p}-\mathbf{q})\cdot\mathbf{w} \\ &\leq \sum_{i=1}^{n} |(\mathbf{v}[\mathrm{i}] - \mathbf{w}[\mathrm{i}])| + (\mathbf{p}-\mathbf{q})\cdot\mathbf{w} \\ &\leq \xi + (\mathbf{p}-\mathbf{q})\cdot\mathbf{w}\end{aligned}$$

□

In GT-CFR, the depth-limited public tree used for search may change at each iteration. Let $\mathcal{L}^t$ be the public tree at time t. For any given tree $\mathcal{L}$, let $\mathcal{N}(\mathcal{L})$ be the interior of the tree: all non-leaf, non-terminal public states. The interior of the tree is where regret matching is used to generate a policy, with regrets stored for all information states in interior public states. Let $\mathcal{F}(\mathcal{L})$ be the frontier of $\mathcal{L}$, containing non-terminal leaves, and $\mathcal{Z}(\mathcal{L})$ be the terminal public states. GT-CFR uses the value function at all public states in the frontier, receiving noisy estimates $\tilde{v}(s)$ of the true counterfactual values $v(s)$. We will distinguish between the true regrets R_s^T computed from the entire policy, and the regret $\tilde{R}_s^T$ computed using the estimated values $\tilde{v}(s)$. Given a sequence of trees across T iterations, let $\mathcal{T}_n(s_{\text{pub}})$ be the set of maximal length intervals $[a, b] \subseteq [1, T]$ where s_{pub} is in $\mathcal{N}(\mathcal{L}^t)$ for all $t \in [a, b]$. Let U be the maximum difference in counterfactual value between any two strategies, at any information state, and A be the maximum number of actions at any information state.

Lemma 3. *After running GT-CFR for T iterations starting at some initial public state s_0, using a value function with quality ϵ, ξ, regret for the strategies satisfies the bound*

$$\sum_{s_i \in \mathcal{S}_i(s_0)} R_{s_i}^T \leq \sum_{t=1}^{T} |\mathcal{F}(\mathcal{L}^t)|(\epsilon + \xi) + \sum_{s_{\text{pub}} \in \bigcup_{t=1}^{T} \mathcal{L}^t} |\mathcal{S}_i(s_{\text{pub}})| U \sqrt{A} \sum_{[a,b] \in \mathcal{T}_n(s_{\text{pub}})} \sqrt{|[a, b]|}$$

Proof. Starting with the definition of regret, and noting that regrets are independently maximised in a perfect recall game, we can rearrange terms to get

$$\begin{aligned}
\sum_{s_i \in \mathcal{S}_i(s_0)} R_{s_i}^T &= \sum_{s_i \in \mathcal{S}_i(s_0)} \left(\max_{\pi_i^*} \sum_{t=1}^{T} v^{\pi^t \leftarrow \pi_i^*}(s_i) - \sum_{t=1}^{T} v^{\pi^t}(s_i) \right) \\
&= \max_{\pi_i^*} \sum_{s_i \in \mathcal{S}_i(s_0)} \left(\sum_{t=1}^{T} v^{\pi^t \leftarrow \pi_i^*}(s_i) - \sum_{t=1}^{T} v^{\pi^t}(s_i) \right) \\
&= \max_{\pi_i^*} \sum_{t=1}^{T} \sum_{s_i \in \mathcal{S}_i(s_0)} \left(v^{\pi^t \leftarrow \pi_i^*}(s_i) - v^{\pi^t}(s_i) \right)
\end{aligned}$$

We can rewrite the counterfactual values of information state s_i in terms of the counterfactual

value of leaves and terminals of the tree.

$$= \max_{\pi_i^*} \sum_{t=1}^{T} \left(\sum_{s_{\text{pub}} \in \mathcal{F}(\mathcal{L}^t)} \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*) v^{\pi^t \leftarrow \pi_i^*}(s_i) - P_i(s_i|\pi^t) v^{\pi^t}(s_i) \right) \right.$$
$$\left. + \sum_{s_{\text{pub}} \in \mathcal{Z}(\mathcal{L}^t)} \sum_{z \in I(s_{\text{pub}})} \left(P_i(z|\pi^t \leftarrow \pi_i^*) v^{\pi^t}(z) - P_i(z|\pi^t) v^{\pi^t}(z) \right) \right) \quad (1)$$

Examining part of the first term inside the sum, we can independently maximise the counterfactual values at each information state s_i. As above, this is equivalent to maximising at public state s_{pub}.

$$\sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*) v^{\pi^t \leftarrow \pi_i^*}(s_i) - P_i(s_i|\pi^t) v^{\pi^t}(s_i) \right)$$
$$\leq \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \max_{\pi^{**}} \left(P_i(s_i|\pi_i^*) v^{\pi^t \leftarrow \pi_i^{**}}(s_i) - P_i(s_i|\pi^t) v^{\pi^t}(s_i) \right)$$
$$= \max_{\pi^{**}} \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*) v^{\pi^t \leftarrow \pi_i^{**}}(s_i) - P_i(s_i|\pi^t) v^{\pi^t}(s_i) \right)$$

Given that we individually maximised over each minuend, we satisfy the requirements of Lemma 1. We can then use the value function quality bounds.

$$\leq \max_{\pi^{**}} \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(v^{\pi^t \leftarrow \pi_i^{**}}(s_i) - v^{\pi^t}(s_i) \right)$$
$$+ \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*) v^{\pi^t}(s_i) - P_i(s_i|\pi^t) v^{\pi^t}(s_i) \right)$$
$$\leq \epsilon + \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*) v^{\pi^t}(s_i) - P_i(s_i|\pi^t) v^{\pi^t}(s_i) \right)$$

Up to this point, we have used the true counterfactual values for the current policy profile. At leaves, however, GT-CFR only has access to the value function's noisy estimates of the true values. Applying Lemma 2, we get

$$\leq \epsilon + \xi + \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*) \tilde{v}^{\pi^t}(s_i) - P_i(s_i|\pi^t) \tilde{v}^{\pi^t}(s_i) \right)$$

Placing this back into Equation 1 and collecting ϵ and ξ terms, we have

$$\sum_{s_i \in \mathcal{S}_i(s_0)} R_{s_i}^T \leq \sum_{t=1}^{T} |\mathcal{F}(\mathcal{L}^t)|(\epsilon + \xi) + \max_{\pi_i^*} \sum_{t=1}^{T} \left(\sum_{s_{\text{pub}} \in \mathcal{F}(\mathcal{L}^t)} \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*)\tilde{v}^{\pi^t}(s_i) - P_i(s_i|\pi^t)\tilde{v}^{\pi^t}(s_i) \right) + \sum_{s_{\text{pub}} \in \mathcal{Z}(\mathcal{L}^t)} \sum_{z \in I(s_{\text{pub}})} \left(P_i(z|\pi^t \leftarrow \pi_i^*)v^{\pi^t}(z) - P_i(z|\pi^t)v^{\pi^t}(z) \right) \right)$$

We can rearrange the sums to consider the regret contribution for each public state

$$= \sum_{t=1}^{T} |\mathcal{F}(\mathcal{L}^t)|(\epsilon + \xi) + \max_{\pi_i^*} \sum_{s_{\text{pub}} \in \bigcup_{t=1}^{T} \mathcal{L}^t} \left(\sum_{t \text{ s.t. } s_{\text{pub}} \in \mathcal{F}(\mathcal{L}^t)} \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*)\tilde{v}^{\pi^t}(s_i) - P_i(s_i|\pi^t)\tilde{v}^{\pi^t}(s_i) \right) + \sum_{t \text{ s.t. } s_{\text{pub}} \in \mathcal{Z}(\mathcal{L}^t)} \sum_{z \in I(s_{\text{pub}})} \left(P_i(z|\pi^t \leftarrow \pi_i^*)v^{\pi^t}(z) - P_i(z|\pi^t)v^{\pi^t}(z) \right) \right)$$

As before we can use Lemma 1 to separate out regrets at the interior states in $\mathcal{N} \coloneqq \mathcal{F}(\mathcal{N}(\bigcup_{t=1}^{T} \mathcal{L}^t))$, which always depend only on leaves and terminals. Let $\mathcal{L}'^{,t}$ be $\mathcal{L}^t$ minus all public states in $\mathcal{N}$ and any successor states.

$$\leq \sum_{t=1}^{T} |\mathcal{F}(\mathcal{L}^t)|(\epsilon + \xi) + \sum_{s_{\text{pub}} \in \mathcal{N}} \sum_{[a,b] \in \mathcal{T}_n(s_{\text{pub}})} \sum_{s_i \in s_{\text{pub}}} \tilde{R}_{s_i}^{a,b} + \max_{\pi_i^*} \sum_{s_{\text{pub}} \in \bigcup_{t=1}^{T} \mathcal{L}'^{,t}} \left(\sum_{t \text{ s.t. } s_{\text{pub}} \in \mathcal{F}(\mathcal{L}'^{,t})} \sum_{s_i \in \mathcal{S}_i(s_{\text{pub}})} \left(P_i(s_i|\pi_i^*)\tilde{v}^{\pi^t}(s_i) - P_i(s_i|\pi^t)\tilde{v}^{\pi^t}(s_i) \right) + \sum_{t \text{ s.t. } s_{\text{pub}} \in \mathcal{Z}(\mathcal{L}'^{,t})} \sum_{z \in I(s_{\text{pub}})} \left(P_i(z|\pi^t \leftarrow \pi_i^*)v^{\pi^t}(z) - P_i(z|\pi^t)v^{\pi^t}(z) \right) \right)$$

Note that the states which were separated out are now effectively terminals in smaller trees. We can repeat this process until regrets for all public states have been separated out.

$$\leq \sum_{t=1}^{T} |\mathcal{F}(\mathcal{L}^t)|(\epsilon + \xi) + \sum_{s_{\text{pub}} \in \bigcup_{t=1}^{T} \mathcal{L}^t} \sum_{[a,b] \in \mathcal{T}_n(s_{\text{pub}})} \sum_{s_i \in s_{\text{pub}}} \tilde{R}_{s_i}^{a,b}$$

Finally, from bounds on regret-matching (*32*),

$$\leq \sum_{t=1}^{T} |\mathcal{F}(\mathcal{L}^t)|(\epsilon + \xi) + \sum_{s_{\text{pub}} \in \bigcup_{t=1}^{T} \mathcal{L}^t} |\mathcal{S}_i(s_{\text{pub}})| U \sqrt{A} \sum_{[a,b] \in \mathcal{T}_n(s_{\text{pub}})} \sqrt{|[a,b]|}$$

□

Note that the form of Lemma 3 implies that regret might not be sub-linear if public states are repeatedly added and removed from the lookahead tree. If we only add states and never remove them, however, we get a standard CFR regret bound plus error terms for the value function.

Theorem 3. *Assume the conditions of Lemma 3 hold, and public states are never removed from the lookahead tree. Then*

$$R_i^{T,\textit{full}} \leq \sum_{t=1}^{T} |\mathcal{F}(\mathcal{L}^t)|(\epsilon + \xi) + \sum_{s_{\text{pub}} \in \mathcal{N}(\mathcal{L}^T)} |\mathcal{S}_i(s_{\text{pub}})| U \sqrt{AT}$$

Proof. This follows from Lemma 3, noting that the interior of $\mathcal{L}^t$ monotonically grows over time. □

Self-play Values as Re-solving Constraints

By using a value network in solving, we lose the ability to compute our opponent's counterfactual best response values to our average strategy (*80*). It is easy to track the opponent's average self-play value across iterations of a CFR variant, but using these values as re-solving constraints does not trivially lead to a bound on exploitability for the re-solved strategy. We show here that average CFR self-play values lead to reasonable, controllable error bounds in the context of continual re-solving. We will use $(x)^+$ to mean $\max\{x, 0\}$. For simplicity, we will also assume that the subgame that is being re-solved is in the GT-CFR lookahead tree for all iterations.

Theorem 4. *Assume we have some average strategy $\bar{\pi}$ generated by T iterations of GT-CFR solver using a value function with quality ϵ, ξ, with final lookahead tree $\mathcal{L}^T$ where public states were never removed from the lookahead tree, and a final average regret R_i^T for the player of interest. Further assume that we have re-solved some public subgame S rooted public state s_{pub}, using the average counterfactual values $\bar{v}(s_o) := \frac{1}{T}\sum_{t=1}^{T} v^{\pi^t}(s_o)$ as the opt-out values in the re-solving gadget. Let π^S be the strategy generated from the re-solving game, with some player and opponent average regrets $\bar{R}_i^S$ and $\bar{R}_o^S$, respectively. Then*

$$\begin{aligned} BV_o^{\bar{\pi} \leftarrow \pi^S} - BV_o^{\bar{\pi}} \leq & (\bar{R}_o^S)^+ + (\bar{R}_i^S)^+ + \bar{R}_i \\ & + 2(\max_t |\mathcal{F}(\mathcal{L}^t_{s_{\text{pub}}})|(\epsilon + \xi) + \sum_{s_{\text{pub}} \in \mathcal{N}(\mathcal{L}^T_{s_{\text{pub}}})} |\mathcal{S}_i(s_{\text{pub}})| U \sqrt{\frac{A}{T}}) \end{aligned}$$

Proof. The general outline of the proof has two parts, both asking the question "how much can the opponent best response value increase?" As in Lemma 4 of (*8*), we can consider breaking the error in re-solving opt-out values into separate underestimation and overestimation terms. The first part of this proof is a bound that takes into account the re-solving solution quality, and how much the average values underestimate the best response to the average. This underestimation is bounded by the opponent regret at a subgame, which requires the solving algorithm to have low regret everywhere in the game: low regret for the opponent does not directly imply that the opponent has low regret in portions of the game that they do not play. The second part of the proof is placing a bound on the overestimation, using the player's regret rather than the opponent's regret.

We start by noting that from the opponent player o's point of view, we can replace an information set s_o with a terminal that has utility $BV^{\bar{\pi}}(s_o)$, and the best response utility $\text{BV}_o^{\bar{\pi},s_o \leftarrow BV^{\bar{\pi}}(s_o)}$ in this modified game will be equal to $\text{BV}_o^{\bar{\pi}}$. We can extend this to the entire subgame S, replacing each s_o with a terminal giving the opponent the best response value: $\text{BV}_o^{\bar{\pi},S \leftarrow BV^{\bar{\pi}}(S)} = \text{BV}_o^{\bar{\pi}}$. Using this notation, we can rewrite $\text{BV}_o^{\bar{\pi} \leftarrow \pi^S}$:

$$\begin{aligned} &\text{BV}_o^{\bar{\pi} \leftarrow \pi^S} - \text{BV}_o^{\bar{\pi}} \\ =&\text{BV}_o^{\bar{\pi},S \leftarrow BV^{\pi^S}(S)} - \text{BV}_o^{\bar{\pi}} \end{aligned}$$

Next, note that $BV^{\pi^S}(s_o)$, the opponent's counterfactual best response to the re-solved subgame strategy π^S at any s_o at the root of S, is no greater than the value of $\max\{BV^{\pi^S}(s_o), \bar{v}(s_o)\}$, the value of s_o within the re-solving game before the gadget where the opponent has decision to opt-out for a fixed value $\bar{v}(s_o)$. That is, adding an extra opponent action which terminates the game never decreases the opponent's best response utility. Extending this to the entire subgame S again, we get

$$\begin{aligned} &\text{BV}_o^{\bar{\pi},S \leftarrow BV^{\pi^S}(S)} - \text{BV}_o^{\bar{\pi}} \\ \leq&\text{BV}_o^{\bar{\pi},S \leftarrow \max\{BV^{\pi^S}(S), \bar{v}\}} - \text{BV}_o^{\bar{\pi}} \end{aligned} \tag{2}$$

From Lemma 1 of (*8*), the game value of a re-solving game with opt-out values $\bar{v}(s_o)$ is $U^S_{\bar{v},\bar{\pi}} + \sum_{s_o \in \mathcal{S}_o(s_{\text{pub}})} \bar{v}(s_o)$, for some underestimation error on the opt-out values that is given by

$$U^S_{\bar{v},\bar{\pi}} := \min_{\pi^{*S}} \sum_{s_o \in \mathcal{S}_o(s_{\text{pub}})} (BV^{\bar{\pi} \leftarrow \pi^{*S}}(s_o) - \bar{v}(s_o))^+$$

Given the re-solving regrets, we have $\text{BV}_o^{\pi^S} \leq (\bar{R}_o^S)^+ + (\bar{R}_i^S)^+ + U^S_{\bar{v},\bar{\pi}} + \sum_{s_o \in \mathcal{S}_o(s_{\text{pub}})} \bar{v}(s_o)$. Because $\text{BV}_o^{\bar{\pi},s_o \leftarrow w+\epsilon} \leq \text{BV}_o^{\bar{\pi},s_o \leftarrow w} + \epsilon$ for $\epsilon \geq 0$, we can use this inequality to update Equation 2. That is, there is some per-information-set values ϵ such that $BV^{\pi^S}(\tilde{S}) = \bar{v}(\cdot) + \epsilon$ and $\epsilon \cdot \mathbf{1} \leq (\bar{R}_o^S)^+ + (\bar{R}_i^S)^+ + U^S_{\bar{v},\bar{\pi}}$, so that

$$
\begin{aligned}
&\mathrm{BV}_o^{\bar{\pi},S\leftarrow\max\{BV^{\pi^S}(S),\bar{v}\}} - \mathrm{BV}_o^{\bar{\pi}} \\
=&\mathrm{BV}_o^{\bar{\pi},S\leftarrow\bar{v}+\epsilon} - \mathrm{BV}_o^{\bar{\pi}} \\
\leq&\mathrm{BV}_o^{\bar{\pi},S\leftarrow\bar{v}} + \epsilon\cdot\mathbf{1} - \mathrm{BV}_o^{\bar{\pi}} \\
\leq&\mathrm{BV}_o^{\bar{\pi},S\leftarrow\bar{v}} + (\bar{R}_o^S)^+ + (\bar{R}_i^S)^+ + U_{\bar{v},\bar{\pi}}^S - \mathrm{BV}_o^{\bar{\pi}} \qquad (3)
\end{aligned}
$$

Looking at $U_{\bar{v},\bar{\pi}}^S$, we note that this minimum is no greater than the case when $\pi^* = \bar{\pi}$. The difference $BV^{\bar{\pi}\leftarrow\pi^{*S}}(s_o) - \bar{v}(s_o)$ is the average full counterfactual regret R_{s_o} of strategy $\bar{\pi}$ at s_o. Restricting our attention to $\mathcal{L}_{s_{\text{pub}}}^t$, the portion of the lookahead tree restricted to s_{pub} and its descendants, Theorem 3 gives us a bound on $U_{\bar{v},\bar{\pi}}^S$ and we can update Equation 3

$$
\begin{aligned}
&\mathrm{BV}_o^{\bar{\pi},S\leftarrow\bar{v}} + (\bar{R}_o^S)^+ + (\bar{R}_i^S)^+ + U_{\bar{v},\bar{\pi}}^S - \mathrm{BV}_o^{\bar{\pi}} \\
\leq&\mathrm{BV}_o^{\bar{\pi},S\leftarrow\bar{v}} - \mathrm{BV}_o^{\bar{\pi}} \qquad (4) \\
&+ (\bar{R}_o^S)^+ + (\bar{R}_i^S)^+ \max_t |\mathcal{F}(\mathcal{L}_{s_{\text{pub}}}^t)|(\epsilon+\xi) + \sum_{s_{\text{pub}}\in\mathcal{N}(\mathcal{L}_{s_{\text{pub}}}^T)} |\mathcal{S}_i(s_{\text{pub}})| U\sqrt{\frac{A}{T}}
\end{aligned}
$$

Looking at just the difference in opponent counterfactual best response values, we can again get an upper bound by giving the opponent the choice at all information sets at the root of subgame S of playing a best response against the unmodified strategy $\bar{\pi}$ to get value $BV^{\bar{\pi}}(S)$, or opting out to get value $\bar{v}$.

$$
\begin{aligned}
&\mathrm{BV}_o^{\bar{\pi},S\leftarrow\bar{v}} - \mathrm{BV}_o^{\bar{\pi}} \\
\leq&\mathrm{BV}_o^{\bar{\pi},S\leftarrow\max\{BV^{\bar{\pi}}(S),\bar{v}\}} - \mathrm{BV}_o^{\bar{\pi}} \\
=&(\mathrm{BV}_o^{\bar{\pi},S\leftarrow\max\{BV^{\bar{\pi}}(S),\bar{v}\}} - \bar{v}_o) - (\mathrm{BV}_o^{\bar{\pi}} - \bar{v}_o) \\
\leq&(\mathrm{BV}_o^{\bar{\pi},S\leftarrow\max\{BV^{\bar{\pi}}(S),\bar{v}\}} - \bar{v}_o) - (\mathrm{BV}_o^{\pi^*} - \bar{v}_o) \\
=&(\mathrm{BV}_o^{\bar{\pi},S\leftarrow\max\{BV^{\bar{\pi}}(S),\bar{v}\}} - \bar{v}_o) - (v_o^{\pi^*} - \bar{v}_o) \\
=&(\mathrm{BV}_o^{\bar{\pi},S\leftarrow\max\{BV^{\bar{\pi}}(S),\bar{v}\}} - \bar{v}_o) + (v_i^{\pi^*} - \bar{v}_i) \\
\leq&(\mathrm{BV}_o^{\bar{\pi},S\leftarrow\max\{BV^{\bar{\pi}}(S),\bar{v}\}} - \bar{v}_o) + (\mathrm{BV}_i^{\bar{\pi}} - \bar{v}_i) \\
=&(\mathrm{BV}_o^{\bar{\pi},S\leftarrow\max\{BV^{\bar{\pi}}(S),\bar{v}\}} - \bar{v}_o) + \bar{R}_i \qquad (5)
\end{aligned}
$$

The difference of the first two terms is the regret in the opt-out game game described above, where we have lifted each iteration strategy π^t into this game by never selecting the opt-out choice. Consider the immediate counterfactual regret $\tilde{R}^T(s_o)$ in this situation for any information state s_o in this augmented game. Writing this in terms of the original immediate counterfactual regret $R^T(s_o)$ and the opt-out value, we get

$$
\begin{aligned}
\tilde{R}^T(s_o) &= \max\{T(\bar{v}[s_o] - \bar{v}[s_o]), R^T(s_o)\} \\
&= (R^T(s_o))^+
\end{aligned}
$$

Because the positive immediate regret in the opt-out game is the same as the positive regret in the original game, we can use the Theorem 3 bound, which is composed from immediate regrets. Putting this together with Equation 4 and Equation 5, we get

$$\begin{aligned}
&\text{BV}_o^{\bar{\pi} \leftarrow \pi^S} - \text{BV}_o^{\bar{\pi}} \\
\leq &(\bar{R}_o^S)^+ + (\bar{R}_i^S)^+ + \bar{R}_i \\
&+ 2(\max_t |\mathcal{F}(\mathcal{L}^t_{s_{\text{pub}}})|(\epsilon + \xi) + \sum_{s_{\text{pub}} \in \mathcal{N}(\mathcal{L}^T_{s_{\text{pub}}})} |\mathcal{S}_i(s_{\text{pub}})| U \sqrt{\frac{A}{T}})
\end{aligned}$$

□

Continual Re-solving

Continual re-solving puts GT-CFR together with re-solving the previously solved subgame. A bound on final solution quality follows directly from applications of Theorem 3 and Theorem 4.

Theorem 5. *Assume we have played a game using continual re-solving, with one initial solve and D re-solving steps. Each solving or re-solving step finds an approximate Nash equilibrium through T iterations of GT-CFR using a value function with quality ϵ, ξ, public states are never removed from the lookahead tree, the maximum interior size $\sum_{s_{\text{pub}} \in \mathcal{N}(\mathcal{L}^T)} |\mathcal{S}_i(s_{\text{pub}})|$ of all lookahead trees is bounded by N, the sum of frontier sizes across all lookahead trees is bounded by F, the maximum number of actions at any information sets is A, and the maximum difference in values between any two strategies is U. The exploitability of the final strategy is then bounded by $(5D + 2)\left(F(\epsilon + \xi) + NU\sqrt{\frac{A}{T}}\right)$.*

Proof. The exploitability EXP_0 of the player's initial strategy from the original solve is bounded by the sum of the regrets for both players. Theorem 3 provides regret bounds for GT-CFR, so

$$\text{EXP}_0 \leq 2\left(F(\epsilon + \xi) + NU\sqrt{\frac{A}{T}}\right)$$

Each subsequent re-solve is operating on the strategy of the previous step, using the average values for the opt-out values. That is, the first re-solve will be updating the strategy from the initial solve, the second re-solve will be updating the subgame strategy from the first re-solve, and so on. Theorem 4 provides a bound on how much the exploitability increases after each re-solving step, with Theorem 3 providing the necessary regret bounds

$$\begin{aligned}
\text{EXP}_d \leq &\text{EXP}_{d-1} + (\bar{R}_o^S)^+ + (\bar{R}_i^S)^+ + \bar{R}_i + 2\left(F(\epsilon + \xi) + NU\sqrt{\frac{A}{T}}\right) \\
\leq &\text{EXP}_{d-1} + 5\left(F(\epsilon + \xi) + NU\sqrt{\frac{A}{T}}\right)
\end{aligned}$$

Unrolling for D re-solving steps leads to the final bound. □

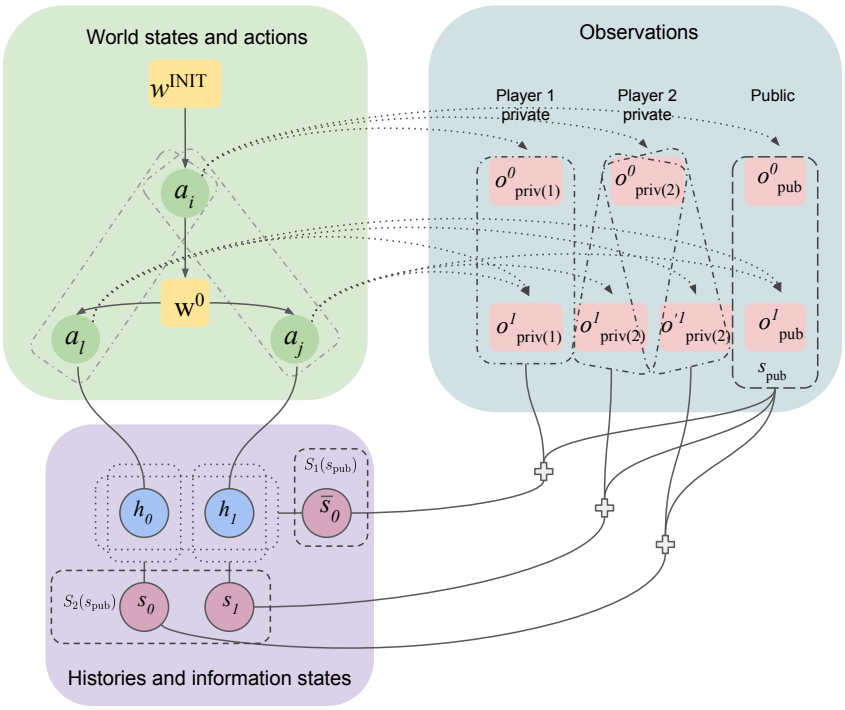


Figure S1: An example of a Factored-Observation Stochastic Game (FOSG). This figure presents the visual view of notation from Background and Terminology. In this example the game starts in w^{init} which is the complete state of the environment containing private information for both players. After playing action a_i the state moves to w^0 where there are two possible actions. Each action emits private and public observations. In this example, actions a_j and a_l emit the same private observation $o^1_{\text{priv}(1)}$ for player 1, therefore they cannot distinguish which action happened. On the other hand, player 2 has different observations $o^1_{\text{priv}(2)}$ and $o'^1_{\text{priv}(2)}$ for each of the actions, therefore they have more information about the state of the environment than player 1. The sequence of public observations shared by both players information is denoted as s_{pub}. Both sequences of actions and factored observations meet in the final 'Histories and information states' view. The two possible action sequences are represented by histories h_0 and h_1, where $h_0 = (a_i, a_l)$, $h_1 = (a_i, a_j)$. Since both actions a_l and a_j result in the same observation for player 1, they cannot tell which one of the histories happened and his information state $\bar{s}_0$ contains them both. This is not the case for player 2, who can separate the histories, and each of his information states s_0 and s_1 contains just one history.

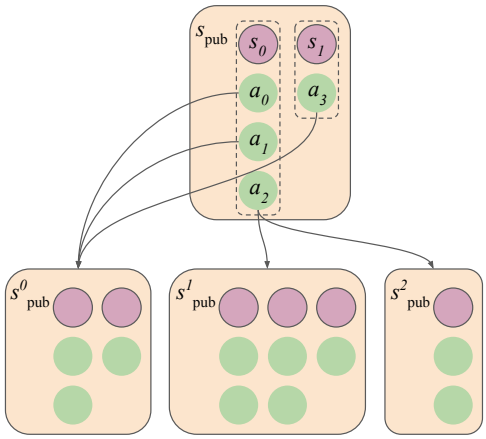


Figure S2: An example of a public tree. The public tree provides different view of the FOSG. In this example actions a_0 and a_1 emit the same public observation and therefore they lead to the same public tree node s^0_{pub}. On the other hand, action a_2 can lead to multiple possible states: for instance when a detective in Scotland Yard moves to a location the game can either 1) end because Mr. X was there and he was caught or 2) it continues because he was in a different station.

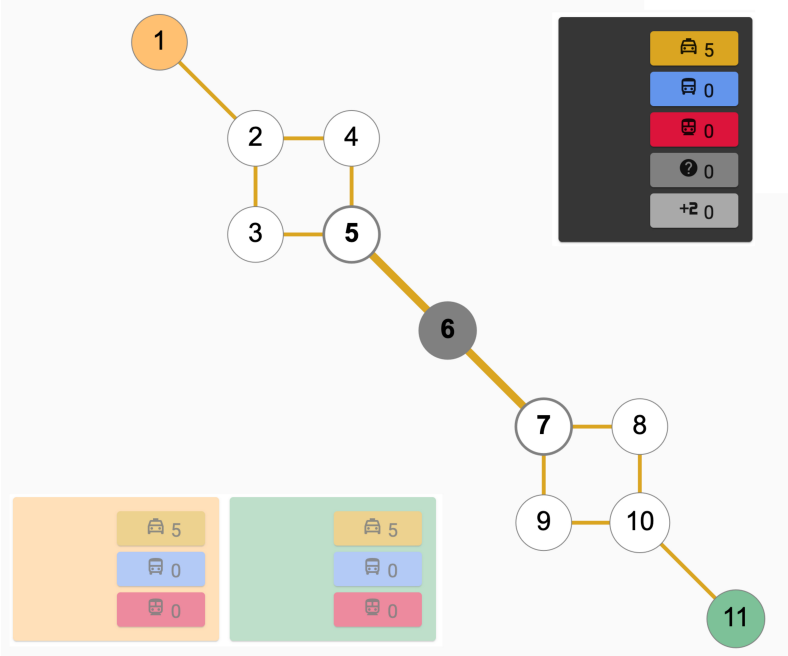
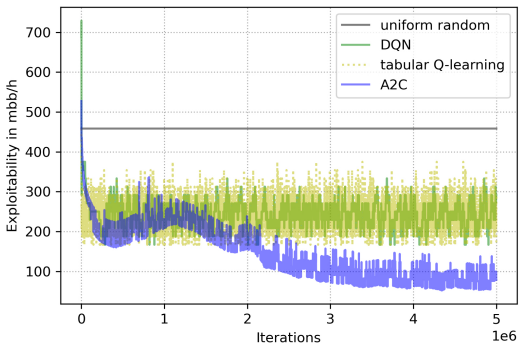
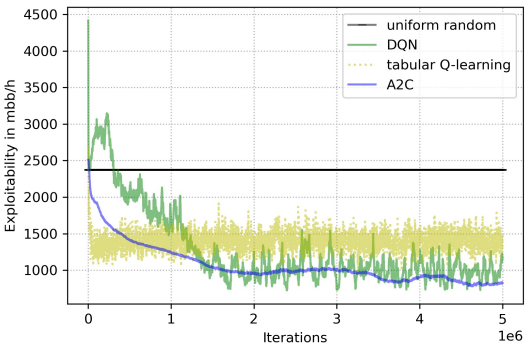


Figure S3: Initial situation on the glasses map for Scotland Yard. Mr. X starts at station 6 while the two detectives start at stations 1 and 11. All of them have 5 taxi cards (all edges in this map are of the same type) and the game is played for 5 rounds.



(**A**) Exploitability in 2-player Kuhn poker



(**B**) Exploitability in 2-player Leduc poker

Figure S4: Comparing performance of DQN, A2C, tabular Q-learning and uniform random policy in (A) Kuhn poker and (B) Leduc poker.

Game	Architecture	Belief features	Public state features
Chess	ResNet	Redundant — there is no uncertainty over players state.	One 8x8 plane for each piece type (6) of each player (2) and repetitions planes (2) for last eight moves + scalar planes (7), 119 8x8 planes in total.
Go	ResNet	Redundant	One 19x19 plane for stones of each player (2) for last eight moves plus a single plane encoding player to act, 17 19x19 planes in total.
Poker	MLP 6 x 2048	1326 (possible private card combinations) * 2 (num of players).	N hot encoding of board cards (52) + commitment of each player normalized by his stack (2) + 1 hot encoding of who acts next, including chance player (3).
Scotland Yard	MLP 6 x 512	1 (detectives' position is always certain) + 199 (possible Mr X's position).	1 hot encoding of position of each detective (5*199) + cards of each detective (5*3) + cards of Mr X (5) + who is playing next (6) + was double move just used (1) + how many rounds were played (1).

Table S1: A neural network architecture and features used for each game.

Hyperparam	Symbol	Chess	Go	Scot. Yard	HUNL
Batch size		2048	2048	1024	1024
Optimizer		sgd	sgd	sgd	adam
Initial learning rate (LR)	α_{init}	0.1	0.02	0.1	0.0001
LR decay steps	T_{decay}	40k	200k	2M	2M
LR decay rate	d	0.8	0.1	0.5	0.5
Policy head weight	w_p	1	1	0.05	0.01
Value head weight	w_v	0.25	0.5	1	1
Replay buffer size		50M	50M	1M	1M
Max grad updates per example		1	0.2	5	10
TD(1) target sample probability	p_{td1}	0	0.2	0	0
Queries per search	q_{search}	1	0	0.3	0.9
Recursive queries per search	$q_{recursive}$	0.2	0	0.1	0.1
Self-play uniform policy mix	ϵ	0	0	0	0.1
Resign enabled		True	True	False	False
Resign threshold	$resign_threshold$	-0.9	-0.9	-	-
Min ratio of games without resign	p_{no_resign}	0.2	0.2	-	-
Greedy play after move	$moves_{greedy_after}$	30	30	never	never
Max moves in one episode	$moves_{max}$	512	722	unlim.	unlim.
Prior softmax temperature	T_{prior}	1.5	1.5	1	1

Table S2: Hyperparameters for each game.

Agent	Rel. Elo
AlphaZero(s=16k, t=800k)	+3139
AlphaZero(s=16k, t=400k)	+3021
AlphaZero(s=8k, t=800k)	+2875
AlphaZero(s=8k, t=400k)	+2801
AlphaZero(s=4k, t=800k)	+2643
AlphaZero(s=16k, t=200k)	+2610
AlphaZero(s=4k, t=400k)	+2584
AlphaZero(s=2k, t=800k)	+2451
AlphaZero(s=8k, t=200k)	+2428
AlphaZero(s=2k, t=400k)	+2353
AlphaZero(s=4k, t=200k)	+2234
AlphaZero(s=800, t=800k)	+2099
AlphaZero(s=16k, t=100k)	+2088
AlphaZero(s=2k, t=200k)	+2063
AlphaZero(s=800, t=400k)	+2036
SoG(s=16k, c=10)	**+1970**
AlphaZero(s=8k, t=100k)	+1940
SoG(s=8k, c=10)	**+1902**
AlphaZero(s=800, t=200k)	+1812
SoG(s=4k, c=10)	**+1796**
AlphaZero(s=4k, t=100k)	+1783
SoG(s=2k, c=10)	**+1672**
AlphaZero(s=2k, t=100k)	+1618
SoG(s=800, c=1)	**+1426**
AlphaZero(s=800, t=100k)	+1360
Pachi(s=100k)	+869
Pachi(s=10k)	+231
GnuGo(l=10)	+0

Table S3: Full Go results (Non Recursive Queries). Elo of GnuGo with a single thread and 100ms thinking time was set to be 0. AlphaZero(s=16k, t=800k) refers to 16000 search simulations after 800000 training steps.

Agent	Rel. Elo
AlphaZero(s=16k, t=800k)	+3431
AlphaZero(s=16k, t=400k)	+3319
AlphaZero(s=8k, t=800k)	+3169
AlphaZero(s=8k, t=400k)	+3093
AlphaZero(s=4k, t=800k)	+2933
AlphaZero(s=16k, t=200k)	+2899
AlphaZero(s=4k, t=400k)	+2880
AlphaZero(s=2k, t=800k)	+2745
AlphaZero(s=8k, t=200k)	+2712
AlphaZero(s=2k, t=400k)	+2643
AlphaZero(s=4k, t=200k)	+2509
AlphaZero(s=800, t=800k)	+2394
AlphaZero(s=16k, t=100k)	+2391
AlphaZero(s=2k, t=200k)	+2348
AlphaZero(s=800, t=400k)	+2315
AlphaZero(s=8k, t=100k)	+2240
AlphaZero(s=800, t=200k)	+2105
AlphaZero(s=4k, t=100k)	+2078
SoG(s=16k, c=10)	**+2025**
SoG(s=8k, c=10)	**+1937**
AlphaZero(s=2k, t=100k)	+1928
SoG(s=4k, c=10)	**+1838**
SoG(s=2k, c=10)	**+1766**
AlphaZero(s=800, t=100k)	+1644
SoG(s=800, c=1)	**+1579**
Pachi(s=100k)	+958
Pachi(s=10k)	+227
GnuGo(l=10)	+0

Table S4: Full Go results (Recursive Queries). Elo of GnuGo with a single thread and 100ms thinking time was set to be 0. AlphaZero(s=16k, t=800k) refers to 16000 search simulations after 800000 training steps.

Num. Sims	UCT const. (C)	Expl. (mvd)	Expl. (mvis)	Expl. (mval)
10	1.0	2168	2449	2173
10	2.0	2058	2408	2341
10	5.0	1902	2615	2517
10	10.0	1738	2555	2360
10	13.0	1799	2517	2598
10	20.0	1821	2830	2349
10	26.0	1888	2861	2669
100	1.0	1489	1509	1333
100	2.0	1404	1587	1395
100	5.0	1239	1145	1094
100	10.0	1213	1195	1245
100	13.0	1218	1292	1227
100	20.0	1350	1456	1342
100	26.0	1448	1747	1568
1000	1.0	1323	1218	1177
1000	2.0	1069	1212	864
1000	5.0	699	778	681
1000	10.0	697	601	632
1000	13.0	741	759	744
1000	20.0	859	962	991
1000	26.0	966	1029	1057
10000	1.0	1348	948	1134
10000	2.0	911	877	763
10000	5.0	516	582	538
10000	10.0	490	485	480
10000	13.0	511	465	470
10000	20.0	572	505	505
10000	26.0	631	575	570

Table S5: Average exploitability (in mbb/h) over five policy constructions obtained by independent searches of IS-MCTS runs at each information state in Leduc Poker. The parameter C is the value of the UCT exploration constant. The final policy is obtained either by normalizing the visit counts (mvd), choosing the action with maximum visits (mvis), or choosing the action with the maximal Monte Carlo value estimate (mval).

Parameter	**DQN**	**Tabular Q-Learning**	**A2C**
Learning rate (lr)	1e-1, 1e-2, 1e-3, 1e-4	NA	Actor lr: 1e-3, 1e-4, 1e-5 Critic lr: 1e-2, 1e-3
Decaying exploration rate	1., 0.8, 0.5, 0.2, 0.1	NA	NA
Replay buffer size	100, 1000, 10000, 100000	NA	NA
Hidden layer size	'32', '64', '128', '32, 32', '64, 64'	NA NA	'32', '64', '128', '32, 32', '64, 64'
Num. of critic updates before every actor update	NA	NA	4, 8, 16
Step size	NA	0.1, 0.2, 0.5, 0.8, 1.0	

Table S6: Hyper parameters swept over in each RL algorithm.